

DON BOSCO INSTITUTE OF MANAGEMENT STUDIES AND COMPUTER APPLICATION

Kumbalagodu, Mysore Road, Bengaluru-560074



A Project Report

ON

**MULTI-TENANT SaaS PLATFORM
(2025-2026)**



BANGALORE UNIVERSITY (JNANABHARATHI)

UNDER THE GUIDANCE OF

**Ms. DAKSHAYINI.L
Asst.professor,
Dept. of BCA, DBIMSCA.**

**SUBMITTED BY
SREESHAN D GOWDA
(U03CQ23S0054)**

BANGALORE UNIVERSITY (JNANABHARATHI)



DON BOSCO INSTITUTE OF MANAGEMENT STUDIES AND COMPUTER APPLICATIONS



This is to certify that **SREESHAN D GOWDA** bearing reg.no **U03CQ23S0054** has successfully completed the project entitled “**MULTI-TENANT SaaS PLATFORM**” under the guidance of **Ms.DAKSHAYINI. L**, for the partial fulfilment of **BCA Sixth semester** course in the computer laboratory at **DONBOSCO INSTITUTE OF MANAGEMENT STUDIES AND COMPUTER APPLICATIONS**, during the period of February 2026 to May 2026.

Project Guide
Asst.Prof Dakshayini L

HOD
Deepa. T

Director
Dr. Veena

Principal
H.Pandurangappa

Signature of the Examiners:

1)

2)

DON BOSCO INSTITUTE OF MANAGEMENT STUDIES AND COMPUTER APPLICATIONS

Kumbalagodu, Mysore Road, Bengaluru-560074



Student Declaration

I **SREESHAN D GOWDA**, hereby declare that this project report entitled “**MULTI-TENANT SaaS PLATFORM**” for the partial fulfilment of **BCA Sixth semester** course in the computer laboratory at **DONBOSCO INSTITUTE OF MANAGEMENT STUDIES AND COMPUTER APPLICATIONS**, during the period of February 2026 to May 2026.

Signature

SREESHAN D GOWDA

ACKNOWLEDGEMENT

I hereby acknowledge, with regards and respects, for the encouragement and guidance extended by **Ass.tProf. Dakshayini L**, faculty, Dept. of Computer Applications, DBIMSCA, Bangalore University, Jnana Bharathi, Bengaluru.

I use this opportunity to thank **Professor Deepa T**, HOD, **Dr. Veena**, Director, Dept. of Computer Applications, DBIMSCA, Bangalore University, especially for providing the platform to develop skills of presentation of Synopsis and Project.

Finally, I would like to thank our Principal **H Pandurangappa** , DBIMSCA, Bangalore University and teachers of the department for their support.

Project Associate: SREESHAN D GOWDA

TABLE OF CONTENTS

Topic Description	Page Range
CHAPTER 1 INTRODUCTION	1
1.1 PROJECT AIMS AND OBJECTIVES	1 - 4
1.2 BACKGROUND OF PROJECT	5 - 8
1.3 PROJECT REQUIREMENTS	9 - 15
CHAPTER 2 SYSTEM REQUIREMENT ANALYSIS	16
2.1 SOFTWARE REQUIREMENT SPECIFICATION	16 - 20
2.2 EXISTING VS PROPOSED SYSTEM	21 - 24
2.3 SOFTWARE TOOL USED	25 - 32
CHAPTER 3 SYSTEM DESIGN	33
3.1 TABLE DESIGN	33 - 43
3.2 DATA FLOW DIAGRAM'S	44 - 51
CHAPTER 4 MODULES DESCRIPTION	52
4.1 OVERVIEW OF SYSTEM MODULES	52
4.2 AUTHENTICATION AND AUTHORIZATION MODULE	52
4.3 TENANT ONBOARDING & AI GENERATION MODULE	53
4.4 ADMIN DASHBOARD AND MANAGEMENT MODULE	53
4.5 DYNAMIC SITE RENDERING MODULE	54
4.6 CUSTOMER CHECKOUT AND ORDER PROCESSING MODULE	54 - 55
CHAPTER 5 IMPLEMENTATION	56
5.1 ENVIRONMENT SETUP AND CONFIGURATION	56
5.2 BACKEND IMPLEMENTATION (EXPRESS & MONGODB)	57
5.3 FRONTEND IMPLEMENTATION (REACT & VITE)	58

5.4 UI AND STYLING IMPLEMENTATION	59
CHAPTER 6 TESTING	60
6.1 TEST CASES	60 - 76
CHAPTER 7 SNAPSHOTS	77
7.1 SCREENSHOTS	77 - 86
CHAPTER 8 FUTURE ENHANCEMENTS	87
8.1 FUTURE ENHANCEMENTS	87 - 88
8.2 SCALABILITY AND DEPLOYMENT PLAN	89 - 90
CHAPTER 9 CONCLUSION	91 - 92
CHAPTER 10 REFERENCES	93 - 94

CHAPTER 1

INTRODUCTION

1.1 PROJECT AIMS AND OBJECTIVES

1.1.1 Primary Objective

The primary objective of this project is to design, develop, and deploy "BizWiz," a comprehensive multi-tenant Software-as-a-Service (SaaS) web application that enables small and medium-sized businesses to create, customize, and manage their own professional digital storefronts and service portals from a single, unified platform. The system is built upon a modern full-stack JavaScript architecture, utilizing React.js for the frontend user interface, Express.js as the backend REST API framework, and MongoDB Atlas as the cloud-hosted NoSQL database. The platform is designed such that a single deployed instance of the application can simultaneously serve an unlimited number of independent business tenants, each with their own unique branding, product catalogues, order management systems, and customer-facing websites — all while maintaining strict logical data isolation between tenants at the database level.

The core philosophy behind BizWiz is to dramatically lower the barrier to entry for establishing a professional online presence. Traditionally, a business owner looking to launch a branded e-commerce store or service booking portal would need to either hire a web development team — costing thousands of rupees and weeks of development time — or rely on rigid marketplace platforms like Amazon or Flipkart where they sacrifice brand identity and customer relationships. BizWiz eliminates both of these problems by offering an intelligent, guided onboarding process that takes a business from zero to a fully functional, publicly accessible website in under five minutes, with no coding or design skills required whatsoever.

Furthermore, the platform integrates Google's Gemini 1.5 Flash generative AI model directly into the onboarding workflow. This AI integration automatically generates professional marketing copy — including compelling hero headlines and informative "About Us" descriptions — tailored specifically to each business's name and industry niche. This

eliminates the need for business owners to possess copywriting skills or hire content writers, further reducing the cost and complexity of launching an online presence.

1.1.2 Specific Goals

The project was developed with the following specific, measurable goals in mind:

- 1. AI-Powered Tenant Onboarding:** The system implements a multi-step onboarding wizard that guides new business owners through a five-stage setup process. In the first step, the user provides their business name and industry niche (e.g., "Vegan Pastries & Coffee"). In the second step, they select a website template from three professionally designed layouts: an E-Commerce/Retail template with product grids, shopping carts, and WhatsApp-based checkout; a Service & Booking template with service listings, pricing cards, and an appointment booking form with date and time selection; or an Event Landing Page template with a bold hero section, features list, and email capture form. In the third step, the user personalizes their site by selecting a brand colour using a colour picker, providing optional contact details such as a support email and store location, and specifying a background image URL. Upon completing this step, the system invokes the Google Gemini 1.5 Flash AI model through a dedicated backend API endpoint (`/api/ai/generate`). The AI analyses the business name and niche and automatically generates a compelling hero headline of under ten words and a professional three-sentence "About Us" paragraph. This AI-generated content is presented to the user in the fifth and final review step, where they can freely edit the text before publishing their site. The entire process is designed to transform a non-technical user into a live website owner within minutes.
- 2. Dynamic Multi-Template Site Rendering:** The platform implements a powerful dynamic rendering engine through its `SiteRenderer` component. When an end-user visits a tenant's unique URL — structured as `/ {businessSlug}`, for example `/johns-bakery` — the renderer extracts the slug from the URL parameters using `React Router`'s `useParams` hook, queries the backend API to fetch the complete tenant configuration document from MongoDB, and then conditionally renders one of three distinct React template components — `TemplateA_Ecommerce`, `TemplateB_Booking`, or `TemplateC_Landing` — based on the `themeType` field stored in the tenant's database record. Each template dynamically applies the tenant's chosen `primaryColor`

as inline CSS styles, displays their custom hero headline and about text, and renders their specific product or service inventory. The renderer also handles SEO optimization by dynamically updating the browser's document title, injecting meta description tags, and updating the favicon link based on the tenant's stored metadata. Additionally, the renderer automatically tracks page views by sending a POST request to the analytics endpoint (`/api/tenant/slug/:slug/view`) each time the site is loaded, which atomically increments the pageViews counter in the database using MongoDB's \$inc operator.

3. **Comprehensive Admin Dashboard:** Each authenticated tenant is provided with a full-featured administrative dashboard accessible at `/admin/dashboard`. This dashboard serves as the central command centre for managing every aspect of their digital storefront. The sidebar navigation provides access to seven distinct sections: My Websites (listing all tenant sites the user owns), Dashboard Overview (displaying key metrics such as current theme type, total items, and total orders), Analytics (showing total page views, total orders, and total revenue from delivered orders), Services/Products (for managing the product or service catalogue), Orders (for viewing and processing incoming customer orders), Coupons (for creating and managing promotional discount codes), and Content & Settings (for customizing the site's theme colour, font family, button shape, colour mode, business details, about text, working hours, SEO metadata, and background image). The dashboard provides complete CRUD (Create, Read, Update, Delete) operations for products and services, allowing tenant admins to add items with titles, descriptions, prices, image URLs, categories, and stock counts. The order management section displays all incoming orders in a detailed table format showing customer name, contact information, delivery address, ordered items with quantities, payment method, order status, total price, and order date. Admins can mark orders as Delivered or Cancelled directly from this table, and can export the complete order history as a CSV file for external record-keeping and accounting purposes.
4. **Secure Authentication with Google OAuth 2.0:** The system implements a modern, passwordless authentication flow using Google OAuth 2.0 for user identity verification, combined with JSON Web Tokens (JWT) for session management. When a user clicks "Sign in with Google" on the login page, the `@react-oauth/google` library

issues a credential token. This token is sent to the backend endpoint `/api/auth/google`, where it is verified against Google's servers using the `google-auth-library` npm package. Upon successful verification, the backend either creates a new user record in the MongoDB users collection (storing the user's name, email, Google ID, and profile picture) or retrieves the existing one and updates their profile picture if it has changed. The backend then signs a custom JWT containing the user's database ID, email, name, and profile picture URL, with a seven-day expiration period, using the HMAC SHA256 algorithm via the `jsonwebtoken` npm package. This JWT is returned to the frontend, stored in the browser's `localStorage`, and automatically attached to all subsequent API requests through an Axios request interceptor configured in the centralized API client (`/src/api/client.js`). The frontend `AuthContext` provider manages global authentication state, performs automatic session restoration on page reload by decoding and validating the stored JWT using the `jwt-decode` library, and provides protected route functionality through a `ProtectedRoute` component that renders an Outlet for authenticated users or redirects unauthenticated users to the login page.

1.1.3 Target Audience and Beneficiaries

The BizWiz platform is designed to serve several distinct user groups, each of whom derives specific benefits from the system:

1. **Small and Medium-Sized Enterprises (SMEs):** Independent retailers, local shop owners, and small-scale manufacturers who require a branded digital storefront to showcase and sell their products online. These businesses typically operate with limited budgets and cannot justify the expense of custom web development, which can range from ₹50,000 to ₹5,00,000 for a basic e-commerce website. BizWiz provides them with a feature-rich alternative at zero upfront cost, offering product management, shopping cart functionality, order tracking, coupon systems, and analytics out of the box.
2. **Service-Based Professionals and Freelancers:** Independent professionals such as photographers, tutors, consultants, salon owners, fitness trainers, and event planners who need a professional booking portal where potential clients can browse their service offerings, view pricing, and schedule appointments. The Service & Booking template (`TemplateB_Booking`) is specifically designed for this audience, offering

dedicated date and time selection fields integrated into the checkout flow, along with configurable working hours that the tenant can set from their dashboard.

3. **Event Organisers and Campaign Managers:** Individuals or organisations looking to create focused, single-page landing sites for events, product launches, or promotional campaigns. The Event Landing Page template (TemplateC_Landing) provides a conversion-optimized layout with a bold hero section, feature highlights, and a call-to-action, ideal for time-bound marketing initiatives that require rapid deployment and maximum visual impact.
4. **Non-Technical Users:** Perhaps the most important beneficiary group — business owners who possess strong domain expertise in their respective industries but lack any technical knowledge of web development, server management, or database administration. The AI-powered onboarding wizard, which automatically generates professional website content using the Google Gemini language model, ensures that these users can create a polished, market-ready website without writing a single line of code, making a single design decision beyond choosing a colour, or composing a single paragraph of marketing copy.

1.2 BACKGROUND OF PROJECT

1.2.1 Evolution of SaaS (Software as a Service)

The software industry has undergone a fundamental transformation over the past two decades, shifting from a product-centric model to a service-centric delivery paradigm. In the early era of computing, software was distributed as physical media — floppy disks and later CDs — that required manual installation on individual machines. Each user was responsible for their own hardware procurement, operating system compatibility checks, and software updates. This model was inherently expensive, difficult to maintain, and created significant barriers to adoption for smaller organisations that lacked dedicated IT departments.

The advent of cloud computing in the mid-2000s introduced the concept of Software-as-a-Service (SaaS), where applications are hosted centrally on cloud infrastructure and accessed by end-users over the internet through a web browser. Pioneered by companies like Salesforce for Customer Relationship Management, Google for productivity tools through Gmail and Google Workspace, and later Shopify for e-commerce, the SaaS model

fundamentally altered the economics of software consumption. Instead of paying a large upfront licence fee and bearing the burden of infrastructure management, users could subscribe to a service on a monthly or annual basis, with the provider handling all aspects of hosting, security, updates, and scalability.

According to industry analysis, the global SaaS market has grown from approximately \$31 billion in 2015 to over \$300 billion in 2025, reflecting a compound annual growth rate of over 25 percent. This explosive growth is driven by several key advantages that the SaaS model offers: reduced total cost of ownership for end-users, automatic updates and feature rollouts that eliminate manual patch management, elastic scalability to handle varying workloads without over-provisioning hardware, universal accessibility from any internet-connected device regardless of operating system, and built-in redundancy and disaster recovery through cloud hosting providers like Amazon Web Services (AWS), Google Cloud Platform (GCP), and Microsoft Azure.

The evolution of web technologies has further accelerated SaaS adoption. Modern JavaScript frameworks like React.js enable the creation of rich, responsive single-page applications (SPAs) that provide desktop-like experiences within a browser window. Cloud-native databases like MongoDB Atlas offer fully managed data persistence with automatic failover and scaling. Together, these technologies have made it feasible for small development teams to build and deploy SaaS platforms that rival enterprise-grade commercial products in both functionality and reliability.

1.2.2 The Need for Multi-Tenant Architectures in Modern Business

Within the SaaS ecosystem, architecture design plays a critical role in determining the platform's efficiency, cost-effectiveness, and scalability. Two primary architectural approaches exist for serving multiple customers: single-tenant and multi-tenant.

In a single-tenant architecture, each customer (tenant) receives a dedicated, isolated instance of the application and its supporting infrastructure — a separate server process, a separate database instance, and often a separate codebase deployment. While this provides maximum data isolation and customisation flexibility, it is extraordinarily expensive to scale. If a platform serves 500 businesses, it must maintain 500 separate server instances, 500 separate database connections, and push code updates and security patches to 500 separate deployments individually. This model is sustainable only for enterprise-grade applications

where each client pays a significant premium — often thousands of dollars per month — for dedicated resources.

In contrast, a multi-tenant architecture — the approach adopted by BizWiz — uses a single instance of the application and a single shared database to serve all tenants simultaneously. Each tenant's data is logically isolated within the shared database using unique identifiers. In BizWiz's implementation, the `userId` field in the Tenant schema links each tenant record to its authenticated owner, the unique `businessSlug` field provides public-facing URL isolation (enforced as unique at the MongoDB index level), and all API queries for protected routes include a mandatory `userId` filter condition to prevent cross-tenant data access.

This approach offers profound advantages for a platform like BizWiz. First, infrastructure costs are shared across all tenants, making the platform economically viable even when serving thousands of small businesses that individually generate minimal revenue. Second, code updates, bug fixes, and security patches are deployed once to the single application instance and immediately benefit every tenant on the platform without requiring individual updates. Third, the platform can scale horizontally by simply increasing the capacity of the single application instance — adding more RAM, CPU cores, or deploying behind a load balancer — rather than provisioning entirely new infrastructure for each new customer.

BizWiz specifically implements multi-tenancy at the application level with a shared database model. All tenant documents — including their business configurations, product inventories, customer orders, coupon codes, page view counters, SEO metadata, and working hour schedules — are stored in a single MongoDB `tenants` collection hosted on MongoDB Atlas. Data isolation is enforced through Mongoose queries that always filter by the authenticated user's `userId` (extracted from the JWT by the auth middleware), ensuring that a tenant admin can only ever access and modify their own records through the API.

1.2.3 Problem Statement

Despite the proliferation of digital tools and online marketplaces, a significant proportion of small and medium-sized businesses in India and globally still lack an independent, branded online presence. The challenges they face can be categorised into three major areas:

1. **Prohibitive Cost of Custom Development:** Commissioning a bespoke e-commerce or booking website from a professional development agency typically costs between

₹1,00,000 and ₹10,00,000 depending on complexity, with ongoing annual maintenance fees of ₹20,000 to ₹50,000 for hosting, domain renewal, SSL certificates, and software updates. For a small bakery, a neighbourhood salon, or a freelance photography studio, these figures represent a substantial portion of their annual revenue and are simply not justifiable from a return-on-investment perspective. Even after paying these costs, the business owner remains dependent on the development agency for any content updates, design changes, or bug fixes, creating an ongoing financial and operational dependency.

2. **Technical Complexity of Existing Solutions:** Platforms like WordPress with WooCommerce, or even hosted solutions like Shopify, while more affordable than custom development, still present a steep learning curve for non-technical users. Configuring custom domains, installing and managing SSL certificates, selecting and troubleshooting theme plugins, understanding concepts like hosting environments, PHP versions, database backups, and caching configurations require a level of technical literacy that many small business owners do not possess. This often leads to abandoned setups, half-finished configurations, or poorly maintained websites with broken functionality and outdated content that do more harm than good to the business's online reputation.
3. **Loss of Brand Identity on Marketplaces:** The alternative of listing products on large marketplaces like Amazon, Flipkart, Zomato, or Swiggy provides immediate visibility but comes at the steep cost of brand identity. The business becomes a line item in someone else's catalogue, with minimal control over visual presentation, customer experience, pricing display, or direct customer relationships. Repeat customers develop loyalty to the marketplace platform, not to the individual business, making it extremely difficult for small businesses to build lasting brand equity, collect customer data for marketing, or differentiate themselves from direct competitors listed on the same page. Additionally, marketplace platforms typically charge commissions of 15-30 percent on each sale, significantly eroding already thin profit margins.

BizWiz directly addresses all three of these challenges. It eliminates development costs by providing a free, hosted platform where businesses can create professional storefronts without any upfront investment. It removes technical complexity through an AI-assisted, five-step onboarding wizard that requires nothing more than typing a business name, selecting a

template, and choosing a colour — the AI handles all content generation automatically. And it restores brand identity by giving each tenant their own unique URL (/business-name), a fully customised colour scheme, personalised AI-generated content, and an independent storefront that carries only their brand name, their products, and their visual identity — not a platform's.

1.3 PROJECT REQUIREMENTS

1.3.1 Functional Requirements

The functional requirements define the specific behaviours, actions, and operations that the BizWiz system must support. Each requirement listed below has been directly implemented in the project's codebase and can be verified through the corresponding source code files.

FR-01: User Registration and Authentication — The system must allow new users to register and existing users to log in using their Google account through the OAuth 2.0 protocol. Upon successful authentication via the /api/auth/google endpoint, the system must issue a JSON Web Token (JWT) with a seven-day expiration period, store the user's name, email, Google ID, and profile picture in the MongoDB users collection, and maintain the session across page reloads by persisting the token in the browser's localStorage. The frontend AuthContext component must manage global authentication state and perform automatic session restoration by decoding and validating the stored JWT on each page load.

FR-02: Multi-Step Tenant Onboarding — The system must provide a guided, five-step onboarding wizard that collects information from new business owners. Step 1 must collect the business name and industry niche with mandatory field validation. Step 2 must allow selection of a website layout from three options: E-Commerce/Retail, Service & Booking, or Event Landing Page. Step 3 must allow brand colour selection via a colour picker, along with optional fields for support email, store location, and background image URL. Step 4 must automatically invoke the Google Gemini 1.5 Flash API to generate a hero headline and "About Us" paragraph, displaying a loading animation during processing. Step 5 must present the AI-generated content in editable fields and provide a "Publish Site" button that creates the tenant record in MongoDB and redirects to the admin dashboard.

FR-03: Dynamic Site Rendering — The system must render a fully functional, publicly accessible website for each registered tenant at the URL path /{businessSlug}. The

SiteRenderer component must extract the business slug from the URL, fetch the tenant's complete configuration from the backend, and render the appropriate template component (TemplateA_Ecommerce, TemplateB_Booking, or TemplateC_Landing) with the tenant's specific branding colours, content, and product data. If no matching tenant is found, the system must display a user-friendly "Site Not Found" error page with a link back to the BizWiz homepage.

FR-04: Product and Service Management — Authenticated tenant admins must be able to perform Create, Read, Update, and Delete operations on their product or service inventory through the admin dashboard. Each item record must support the following fields: title, description, price, image URL, category, and stock count (where null represents unlimited stock). These operations must be handled through the REST API endpoints `/api/tenant/:id/items` (POST for creation), `/api/tenant/:id/items/:itemId` (PUT for updates and DELETE for removal).

FR-05: Order Processing and Management — End-customers visiting a tenant's storefront must be able to browse products, add items to a shopping cart, apply coupon codes for discounts, and submit orders with their name, phone number, delivery address, and selected payment method. Upon order submission, the system must automatically decrement the stock count of ordered items and record the order in the tenant's document. Tenant admins must be able to view all incoming orders in a tabular format, update their status through the lifecycle (Pending → Processing → Shipped → Delivered), handle cancellation requests, and export the complete order history as a CSV file.

FR-06: Coupon and Discount Management — The system must allow tenant admins to create promotional coupon codes with configurable percentage discounts ranging from 1% to 100%. Each coupon must have an active/inactive toggle. The public storefront must provide a coupon validation endpoint (`/api/tenant/slug/:slug/validate-coupon`) that checks the entered code against the tenant's active coupons in a case-insensitive manner and returns the applicable discount percentage if valid, or an error message if the coupon is invalid or expired.

FR-07: Analytics and Page View Tracking — The system must automatically track and display page view counts for each tenant's public storefront. When an end-user loads a tenant's site, the SiteRenderer component must send a POST request to

/api/tenant/slug/:slug/view, which atomically increments the pageViews counter using MongoDB's \$inc operator. The admin dashboard's Analytics section must display this metric alongside the total number of orders and the total revenue calculated from delivered orders only.

1.3.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints that the system must satisfy beyond its core functional behaviour.

NFR-01: Security — All user passwords are eliminated from the system entirely by using Google OAuth 2.0 as the sole authentication mechanism, removing the risk of password database breaches. API endpoints that modify tenant data are protected by JWT-based authentication middleware (/backend/middleware/auth.js) that validates the Authorization: Bearer header on every request and rejects any request with a missing, malformed, or expired token with a 401 Unauthorized response. Tenant data isolation is enforced at the query level, where all Mongoose queries for protected routes include { userId: req.user.id } as a filter condition, ensuring that an authenticated user can never access or modify another tenant's data even if they manipulate API request parameters. Cross-Origin Resource Sharing (CORS) is configured on the Express server to accept requests only from the trusted frontend origin (<http://localhost:5173>), preventing unauthorised cross-origin API access. Additionally, the Axios response interceptor on the frontend automatically clears stored credentials from localStorage upon receiving a 401 response, ensuring that expired sessions are cleaned up gracefully.

NFR-02: Scalability — The use of MongoDB Atlas as a cloud-hosted database provides automatic horizontal scaling through sharding and replica sets, allowing the platform to handle a growing number of tenant records without manual infrastructure provisioning. The stateless nature of the JWT-based authentication system means that the backend server does not maintain any server-side session state, allowing multiple server instances to be deployed behind a load balancer without session synchronisation issues. The embedded document pattern used in the Tenant schema, where items, orders, and coupons are stored as sub-documents within the tenant document rather than in separate collections with foreign key references, optimises read performance by reducing the number of database queries required to render a complete tenant storefront to a single MongoDB find query.

NFR-03: Usability — The frontend is built with React.js and styled using Tailwind CSS, providing a fully responsive design that adapts seamlessly to desktop, tablet, and mobile screen sizes through utility-first CSS classes. The onboarding wizard includes a visual progress tracker with five numbered steps and colour-coded completion indicators (blue for current, green for completed, grey for upcoming), inline validation error messages displayed with an AlertCircle icon, and smooth CSS animations (animate-slide-up, animate-fade-in) for step transitions, ensuring that even first-time non-technical users can complete the process without confusion. The admin dashboard uses a sidebar navigation with seven clearly labelled sections accompanied by Lucide React icons, reducing cognitive overload and providing contextual access to relevant features. The platform supports both light and dark colour modes, with a dedicated ThemeToggle component accessible from the onboarding wizard and the dashboard sidebar that persists the user's visual preference.

NFR-04: Performance — The frontend application is built using Vite as the development and build tool, which provides near-instantaneous Hot Module Replacement (HMR) during development and optimised code splitting and tree-shaking for production builds, resulting in fast initial page load times. The centralized Axios client is configured with a 10-second timeout to prevent hanging requests from degrading the user experience. Dynamic site rendering is optimised by fetching the complete tenant document in a single API call, avoiding multiple round-trip requests to the server. The Google Gemini AI integration includes a built-in fallback mechanism — if the AI API fails, returns an unparseable response, or times out, the backend gracefully falls back to pre-written professional template content (e.g., "{businessName} — Excellence You Can Trust") rather than showing an error to the user, ensuring that the onboarding process is never blocked by an external service failure.

NFR-05: Maintainability — The codebase follows a clean separation of concerns. The backend is organised into distinct models/ (Mongoose schemas), routes/ (Express route handlers), and middleware/ (authentication guards) directories. The frontend separates pages/ (full-page components like LandingPage, LoginPage, AdminDashboard), components/ (reusable UI elements like ProtectedRoute, ThemeToggle), templates/ (tenant-facing website layouts), context/ (global state providers like AuthContext), and api/ (the centralized Axios client configuration) into dedicated directories. The use of environment variables managed through the dotenv package and a .env file for sensitive configuration values — including the MongoDB connection URI (MONGODB_URI), Google Client ID (GOOGLE_CLIENT_ID),

JWT secret key (JWT_SECRET), and Gemini API key (GEMINI_API_KEY) — ensures that the codebase can be deployed across different environments (development, staging, production) without any code modifications.

1.3.3 Hardware and Software Requirements

The hardware and software requirements for the BizWiz platform are categorised into the development environment (used during the project's creation), the production/deployment environment (for serving end-users), and the minimum client-side requirements (for accessing the platform).

Table 1.1: Software Requirements — Development Environment

Component	Technology	Version / Details
Operating System	Windows / macOS / Linux	Windows 10/11 recommended
Code Editor	Visual Studio Code	Latest stable version
Runtime Environment	Node.js	v16.0 or higher
Package Manager	npm	v8.0 or higher (bundled with Node.js)
Frontend Framework	React.js	v18.x
Frontend Build Tool	Vite	v5.x
CSS Framework	Tailwind CSS	v3.x
Backend Framework	Express.js	v4.18.x
Database ODM	Mongoose	v8.4.x

Component	Technology	Version / Details
Database	MongoDB Atlas	M0 Free Tier (Cloud-hosted)
AI Service	Google Gemini API	Gemini 1.5 Flash model via @google/generative-ai v0.21.x
Authentication	Google OAuth 2.0	via google-auth-library v9.x and @react-oauth/google
Token Management	JSON Web Tokens	via jsonwebtoken v9.x and jwt-decode
HTTP Client	Axios	Latest version with request/response interceptors
Icon Library	Lucide React	Latest version (SVG-based icons)
Version Control	Git and GitHub	Latest version
API Testing	Postman	Latest version
Web Browser	Google Chrome / Mozilla Firefox	Latest version (for testing and debugging)

Table 1.2: Software Requirements — Production / Deployment Environment

Component	Technology	Details
Frontend Hosting	Vercel	Free tier with automatic HTTPS and global CDN
Backend Hosting	Render / Railway	Free tier with auto-deploy from GitHub
Database	MongoDB Atlas	M0 Free Tier with 512 MB storage and automatic backups

Component	Technology	Details
Domain (Optional)	Custom domain via Vercel	Configurable per deployment

Table 1.3: Minimum Hardware Requirements — Development Machine

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 / AMD Ryzen 5 or higher
RAM	4 GB	8 GB or higher
Storage	10 GB free disk space	20 GB free SSD storage
Internet Connection	Broadband connection	Stable broadband (for MongoDB Atlas, Google APIs, npm)
Display	1366 × 768 minimum resolution	1920 × 1080 or higher

Table 1.4: Minimum Hardware Requirements — End-User / Client

Component	Specification
Device	Any modern smartphone, tablet, or personal computer
Web Browser	Google Chrome, Mozilla Firefox, Apple Safari, or Microsoft Edge (latest version)
Internet Connection	Any active internet connection (3G speed or above)
Minimum Screen Width	320 pixels (responsive design adapts to all sizes)

CHAPTER 2

SYSTEM ANALYSIS

2.1 SOFTWARE REQUIREMENT SPECIFICATION

2.1.1 Introduction to SRS

A Software Requirement Specification (SRS) is a comprehensive document that describes the intended purpose, functionality, and constraints of a software system. It serves as a formal contract between the development team and the stakeholders, ensuring that both parties share a common understanding of what the system will do, how it will behave, and what limitations apply. For the BizWiz multi-tenant SaaS platform, the SRS defines the complete set of requirements governing user authentication, tenant provisioning, content management, dynamic site rendering, order processing, and analytics tracking.

The SRS for BizWiz was developed using the IEEE 830 standard as a guiding framework, which recommends organising requirements into functional requirements (what the system must do), non-functional requirements (quality attributes such as performance, security, and usability), and interface requirements (how the system communicates with external entities and users). The functional and non-functional requirements have been documented in detail in Chapter 1, Section 1.3. This section focuses on elaborating the system's operational environment, user roles, use case specifications, and interface design.

2.1.2 System Overview and Operational Environment

The BizWiz platform operates as a client-server web application deployed across a distributed cloud infrastructure. The operational environment consists of three primary tiers:

1. **Client Tier:** The end-user interacts with the platform through a modern web browser (Google Chrome, Mozilla Firefox, Apple Safari, or Microsoft Edge) on any device — desktop, laptop, tablet, or smartphone. The frontend is a React.js Single Page Application (SPA) built with Vite and styled using Tailwind CSS. It communicates with the backend exclusively through asynchronous HTTP requests using the Axios library.

- 2. **Application Tier:** The backend server runs on Node.js (v16 or higher) using the Express.js web framework. It exposes a RESTful API at the base URL <http://localhost:5000/api> (in development) and handles all business logic including authentication, tenant CRUD operations, AI content generation, order processing, and analytics. The server is stateless — it does not maintain any server-side session data, relying entirely on JWT tokens for authentication.
- 3. **Data Tier:** All persistent data is stored in MongoDB Atlas, a fully managed cloud database service. The database uses two primary collections: users (for authentication records) and tenants (for all business configurations, products, orders, coupons, and analytics data). The Mongoose ODM library enforces schema validation at the application level.

2.1.3 User Roles and Access Levels

The BizWiz platform defines three distinct user roles, each with specific permissions and access boundaries:

Table 2.1: User Roles and Permissions

Role	Description	Accessible Routes	Key Permissions
Unauthenticated Visitor	A user who has not logged in	Landing Page (/), Login Page (/login), Public Tenant Sites (/ {businessSlug})	Can view the BizWiz marketing site; can browse any tenant's public storefront; can place orders and apply coupons on tenant storefronts; cannot access any admin functionality
Authenticated Tenant Admin	A user who has signed in via Google OAuth 2.0	All visitor routes plus Onboarding Wizard (/onboarding), Admin Dashboard (/admin/dashboard)	Can create new tenant websites via the onboarding wizard; can manage their own products, orders, coupons, and site settings; can view analytics for their own sites; can delete their

Role	Description	Accessible Routes	Key Permissions
			own websites; cannot access any other tenant's data
End Customer	A visitor browsing a specific tenant's storefront	Public Tenant Site (/ {businessSlug}) only	Can browse the tenant's product catalogue; can add items to cart; can apply coupon codes; can place orders with delivery details; can view their own order history (stored in browser localStorage); can cancel pending orders

Access control is enforced at two levels. At the frontend level, the ProtectedRoute component checks the AuthContext for a valid user session before rendering protected routes (onboarding and dashboard). If no valid session exists, the user is automatically redirected to /login. At the backend level, the auth middleware (/backend/middleware/auth.js) intercepts all protected API requests, extracts the JWT from the Authorization header, verifies its signature and expiration using the jsonwebtoken library, and injects the decoded user object into the request (req.user). If verification fails, the middleware immediately returns a 401 Unauthorized response.

2.1.4 Use Case Specifications

The following use cases describe the primary interactions between actors and the BizWiz system:

Use Case 1 — User Registration and Login: The actor (Business Owner) navigates to the Login Page and clicks "Sign in with Google." The Google OAuth popup appears, and the user selects their Google account. The frontend receives a credential token from Google and sends it to POST /api/auth/google. The backend verifies the token, creates or retrieves the user record in MongoDB, signs a JWT, and returns it to the frontend. The AuthContext stores the JWT in localStorage and updates the global user state. The user is redirected to the Dashboard if they have existing websites, or to the Onboarding Wizard if they are a new user.

Use Case 2 — AI-Powered Website Creation: The actor (Authenticated Tenant Admin) enters the five-step onboarding wizard. They input their business name and niche (Step 1), select a template type — ecommerce, booking, or landing (Step 2), choose a brand colour and optionally provide contact details (Step 3). The system sends a request to POST `/api/ai/generate` with the business name and niche. The Google Gemini 1.5 Flash AI processes the prompt and returns a JSON object containing a hero headline and about text. The user reviews and optionally edits the AI-generated content (Step 5), then clicks "Publish Site." The system sends a POST request to `/api/tenant` to create the tenant document in MongoDB with a unique business slug derived from the business name.

Use Case 3 — Product/Service Management: The actor (Tenant Admin) navigates to the "Services / Products" tab in the Admin Dashboard. They click "Add New" to open the item creation modal. They fill in the item title, description, price, image URL, and stock count (optional — null means unlimited stock). They submit the form, which triggers a POST request to `/api/tenant/:id/items`. The new item appears in the catalogue list. They can edit any existing item (triggering PUT `/api/tenant/:id/items/:itemId`) or delete it (triggering DELETE `/api/tenant/:id/items/:itemId`).

Use Case 4 — Customer Order Placement: The actor (End Customer) visits a tenant's public storefront at `/ {businessSlug}`. The SiteRenderer fetches the tenant data and renders the appropriate template. On the E-Commerce template, the customer browses the product grid, clicks "Add to Cart" on desired items, adjusts quantities, opens the cart sidebar, optionally applies a coupon code (validated via POST `/api/tenant/slug/:slug/validate-coupon`), proceeds to checkout, fills in their name, phone number, delivery address, and selects a payment method (Cash on Delivery or UPI on Delivery). Upon submission, the order is recorded in the tenant's MongoDB document via POST `/api/tenant/slug/:slug/orders`, stock counts are decremented, and the customer receives an order confirmation with a unique Order ID.

Use Case 5 — Order Management by Admin: The actor (Tenant Admin) navigates to the "Orders" tab in the Dashboard. They see all incoming orders displayed in a table with customer name, contact details, ordered items, payment method, status, total price, and date. They can mark an order as "Delivered" by clicking the check icon, or "Cancelled" by clicking the X icon. These actions trigger PUT `/api/tenant/:id/orders/:orderId/status`. The admin can also export all orders as a CSV file by clicking the "Export CSV" button, which generates a downloadable file containing all order records.

2.1.5 Input and Output Design

System Inputs:

Input	Source	Format	Validation
Google OAuth Credential	Google Identity Services	JWT string	Verified against Google's public keys via google-auth-library
Business Name	Onboarding Wizard Step 1	Text string	Required, non-empty
Business Niche	Onboarding Wizard Step 1	Text string	Required, non-empty
Template Selection	Onboarding Wizard Step 2	Enum: 'ecommerce', 'booking', 'landing'	Must be one of three valid values
Primary Colour	Onboarding Wizard Step 3	Hex colour string (e.g., #3b82f6)	Valid hex format
Product/Service Details	Admin Dashboard Item Modal	JSON: title, description, price, imageUrl, stockCount	Title required, price must be a number
Customer Order Details	Storefront Checkout Form	JSON: name, phone, address, paymentMethod, items, totalPrice	All fields required except couponCode
Coupon Code	Storefront Checkout	Uppercase text string	Case-insensitive match against active coupons

System Outputs:

Output	Destination	Format	Description
JWT Token	Frontend (localStorage)	Signed JWT string	Contains user ID, email, name, picture; expires in 7 days
AI-Generated Content	Onboarding Wizard Step 5	JSON: heroHeadline, aboutText	Generated by Gemini 1.5 Flash; editable by user before publishing
Rendered Tenant Website	Browser (/{businessSlug})	Dynamic HTML/CSS/JS	Template-specific layout with tenant branding, products, and content
Order Confirmation	Storefront Checkout Modal	JSON: orderId, message	Displayed with success animation and order details
Analytics Data	Admin Dashboard Analytics Tab	Numeric values	Page views, total orders, total revenue (delivered orders only)
CSV Export File	Browser Download	.csv file	Contains all order records with headers: Order ID, Customer Name, Phone, Address, Items, Total, Coupon, Status, Date

2.2 EXISTING VS PROPOSED SYSTEM

2.2.1 Drawbacks of the Existing System

Before the development of BizWiz, small and medium-sized businesses seeking to establish an online presence were limited to the following existing approaches, each carrying significant drawbacks:

1. **Custom Web Development:** Hiring a freelance developer or a web development agency to build a bespoke website from scratch. While this produces a fully tailored solution, the typical cost ranges from ₹1,00,000 to ₹10,00,000 depending on complexity, with development timelines of 4 to 12 weeks. Ongoing maintenance, hosting, and updates add ₹20,000 to ₹50,000 annually. This approach is financially prohibitive for small businesses with limited capital and creates a permanent dependency on external technical personnel for even minor content updates.
2. **Drag-and-Drop Website Builders (Wix, Squarespace, WordPress):** These platforms reduce development costs but require significant time investment and a moderate level of technical literacy. Users must navigate complex dashboards, select and customize themes, install plugins for basic functionality like shopping carts or booking forms, configure SSL certificates and domain settings, and write all their own marketing copy. The average time to launch a functional business website using these platforms is 2 to 4 weeks. Additionally, most professional features require paid subscriptions ranging from \$15 to \$50 per month, and the resulting websites are generic in appearance due to over-reliance on pre-built templates.
3. **Marketplace Listings (Amazon, Flipkart, Zomato, Swiggy):** Listing products or services on existing marketplaces provides immediate visibility but at the cost of brand identity, customer relationships, and profit margins. Marketplaces typically charge commissions of 15 to 30 percent per transaction, display competitor products alongside the business's offerings, and retain ownership of customer data and purchase analytics. The business has no control over the visual presentation, user experience, or marketing strategy of their listing.
4. **Social Media Pages (Instagram, Facebook):** Many small businesses use social media profiles as their primary digital presence. While free and easy to set up, social media pages lack essential business functionality such as integrated shopping carts, inventory management, order tracking, coupon systems, and SEO optimization. They

are also subject to platform algorithm changes that can drastically reduce organic reach overnight, and they project an unprofessional image compared to a dedicated business website.

2.2.2 Advantages of the Proposed System (BizWiz)

The BizWiz platform addresses every drawback identified in the existing systems through its innovative multi-tenant SaaS architecture and AI-powered automation:

Table 2.2: Detailed Comparison — Existing Systems vs BizWiz

Parameter	Custom Development	Website Builders (Wix/WordPress)	Marketplaces	BizWiz (Proposed)
Setup Cost	₹1,00,000 – ₹10,00,000	₹1,000 – ₹4,000/month	Free to list	Free
Time to Launch	4 – 12 weeks	2 – 4 weeks	Immediate (no website)	Under 5 minutes
Technical Skills Required	None (developer handles)	Moderate (themes, plugins, DNS)	None	None (AI generates content)
Brand Customization	Full control	Limited to template options	None	Full control (colour, font, button shape, template, content)
Product/Order Management	Custom built (expensive)	Requires plugins	Marketplace controls	Built-in CRUD, order tracking, CSV export
AI Content Generation	Not available	Not available	Not available	Gemini 1.5 Flash generates

Parameter	Custom Development	Website Builders (Wix/WordPress)	Marketplaces	BizWiz (Proposed)
				headlines and about text
Data Ownership	Full	Full (with hosting dependency)	Marketplace owns data	Full — stored in tenant's MongoDB document
SEO Control	Full	Moderate (plugin dependent)	None (marketplace SEO)	Built-in meta title, description, favicon management
Coupon/Discount System	Custom built	Plugin required	Marketplace controls	Built-in coupon creation and validation
Analytics	Custom built or third-party	Plugin required	Marketplace analytics only	Built-in page views, order count, revenue tracking
Maintenance	Ongoing developer dependency	Self-managed (updates, backups)	None required	Zero maintenance — platform handles everything
Scalability	Requires infrastructure investment	Limited by hosting plan	Marketplace scales	Automatic — multi-tenant architecture shares resources

2.2.3 Key Differentiators of BizWiz

The proposed BizWiz system distinguishes itself from all existing solutions through four fundamental innovations:

1. **AI-First Onboarding:** BizWiz is the only solution in this comparison that uses generative artificial intelligence (Google Gemini 1.5 Flash) to automatically create professional marketing copy during the website setup process. The business owner provides only their name and niche, and the AI generates a hero headline and about text specifically tailored to their industry. This eliminates the need for copywriting skills or hiring content writers, which is a barrier that exists across all other solutions.
2. **True Multi-Tenant Architecture:** Unlike custom development (one codebase per client) or website builders (one hosting plan per site), BizWiz runs a single application instance serving all tenants simultaneously. This shared infrastructure model reduces per-tenant costs to near zero, making the platform economically sustainable even when serving thousands of small businesses that individually generate minimal revenue.
3. **Zero-Configuration Deployment:** When a user completes the BizWiz onboarding wizard and clicks "Publish Site," their website is instantly live at a unique URL (`/({businessSlug})`). There is no domain configuration, no hosting setup, no SSL certificate installation, no DNS management, and no server provisioning required. The SiteRenderer component handles everything automatically by dynamically fetching and rendering the tenant's data from the shared MongoDB database.
4. **Integrated Business Operations:** BizWiz provides a complete business management suite — product management, order processing with status tracking, coupon creation and validation, customer analytics, CSV export, and SEO optimization — all within a single unified dashboard. In contrast, existing solutions require multiple plugins, third-party integrations, or separate software subscriptions to achieve the same functionality.

2.3 SOFTWARE TOOL USED

2.3.1 Frontend Technologies

Table 2.3: Frontend Technology Stack

Technology	Version	Purpose	Justification
React.js	v19.2.x	User Interface Library	React's component-based architecture enables extreme code reusability. UI elements like buttons, modals, and form inputs are written once as self-contained components and composed together to build complex interfaces. React's Virtual DOM efficiently calculates the minimal set of real DOM updates required when application state changes, resulting in smooth, fluid user interactions. React is the industry standard for building Single Page Applications (SPAs) with over 40 million weekly npm downloads.
Vite	v8.0.x	Frontend Build Tool	Vite provides near-instantaneous Hot Module Replacement (HMR) during development by serving source code directly using native ES modules, eliminating the slow bundling step required by older tools like Webpack. In production, Vite performs optimised code splitting and tree-shaking, generating minimal JavaScript bundles for fast page loads. Cold start times are typically under 300 milliseconds compared to 10-30 seconds for Webpack-based setups.
Tailwind CSS	v3.4.x	Utility-First CSS Framework	Tailwind provides low-level utility classes that map directly to CSS properties (e.g., px-4 = padding-left/right: 1rem). This is critical for BizWiz because the dynamic theming system needs to compute and inject specific styling based on each tenant's

Technology	Version	Purpose	Justification
			configuration stored in the database. Tailwind's JIT (Just-In-Time) compiler strips unused CSS during production builds, resulting in extremely small final CSS bundles.
React Router DOM	v7.15.x	Client-Side Routing	Enables navigation between pages (Landing, Login, Onboarding, Dashboard, SiteRenderer) without full page reloads. Supports dynamic route parameters (/:businessSlug) which the SiteRenderer uses to determine which tenant's website to render. Uses the HTML5 History API for browser back/forward button support.
Axios	v1.16.x	HTTP Client	Provides a centralised HTTP client with request interceptors that automatically attach the JWT token to every outgoing API request, and response interceptors that clear stored credentials upon receiving a 401 Unauthorized response. Configured with a 10-second timeout to prevent hanging requests.
Lucide React	v1.16.x	SVG Icon Library	Provides a comprehensive set of open-source SVG icons that render as mathematical vector paths rather than raster images. Icons remain razor-sharp on all screen resolutions including high-DPI (Retina) displays while adding negligible weight to the JavaScript bundle.
jwt-decode	v4.0.x	JWT Decoding	Used by the AuthContext to decode the stored JWT on page load, extracting the user's ID, name, email, and profile picture for session restoration without making a network request to the backend.

Technology	Version	Purpose	Justification
@react-oauth/google	v0.13.x	Google OAuth Integration	Provides the "Sign in with Google" button component and handles the OAuth 2.0 flow on the frontend, issuing a credential token that is sent to the backend for verification.

2.3.2 Backend Technologies

Table 2.4: Backend Technology Stack

Technology	Version	Purpose	Justification
Node.js	v16+	Server Runtime Environment	Node.js executes JavaScript on the server side using Google Chrome's V8 engine. Its asynchronous, event-driven, non-blocking I/O model makes it exceptionally suited for handling thousands of concurrent connections typical of a multi-tenant SaaS platform. A single Node.js process can handle 10,000+ concurrent connections with minimal RAM overhead, unlike traditional blocking languages where each connection requires a dedicated thread.
Express.js	v4.18.x	Web Application Framework	Express provides a minimalist abstraction over Node.js's built-in HTTP module, simplifying route definition, middleware management, and request/response handling. Its middleware architecture is used for CORS configuration, JSON body parsing, and JWT authentication verification. Express handles all RESTful API route definitions across three route files: auth.js

Technology	Version	Purpose	Justification
			(authentication), tenant.js (tenant management and orders), and ai.js (AI content generation).
Mongoose	v8.4.x	MongoDB Object Data Modelling (ODM)	Mongoose provides schema-based modelling for MongoDB, enforcing data type validation, required field checks, and default values at the application level. It defines strict schemas for User and Tenant documents, preventing malformed data from entering the database. Features like embedded sub-document schemas (for items, orders, and coupons within the Tenant document) and automatic timestamps simplify complex data operations.
jsonwebtoken	v9.0.x	JWT Token Management	Used to sign JWTs upon successful authentication (with HMAC SHA256 algorithm and 7-day expiration) and to verify incoming tokens in the auth middleware. Provides stateless session management, eliminating the need for server-side session storage.
google-auth-library	v9.14.x	Google OAuth Verification	Verifies Google OAuth credential tokens against Google's public keys on the backend. Ensures the credential was genuinely issued by Google and extracts the user's Google ID, name, email, and profile picture from the verified payload.

Technology	Version	Purpose	Justification
@google/generative-ai	v0.21.x	Google Gemini AI Integration	Provides the SDK for communicating with the Google Gemini 1.5 Flash generative AI model. Used in the /api/ai/generate endpoint to send engineered prompts containing the business name and niche, and to receive JSON-formatted responses with generated hero headlines and about text. Includes a graceful fallback mechanism that returns pre-written template content if the AI API fails.
cors	v2.8.x	Cross-Origin Resource Sharing	Configures the Express server to accept requests only from the trusted frontend origin (http://localhost:5173), preventing unauthorised cross-origin API access from malicious websites.
dotenv	v16.4.x	Environment Configuration	Loads sensitive configuration values (MONGODB_URI, GOOGLE_CLIENT_ID, JWT_SECRET, GEMINI_API_KEY) from a .env file into process.env, ensuring secrets are never hardcoded in the source code repository.

2.3.3 Database Technology

Table 2.5: Database Technology

Technology	Version	Purpose	Justification
MongoDB	v6.x	NoSQL Document Database	MongoDB stores data in flexible, JSON-like BSON documents rather than rigid relational tables. This is critical for BizWiz because different tenants have

Technology	Version	Purpose	Justification
			different data structures — an E-Commerce tenant stores product arrays with prices and stock counts, a Booking tenant stores service arrays with working hours, and a Landing Page tenant stores feature highlight arrays. MongoDB's dynamic schema accommodates all these variations within a single collection without requiring schema migrations.
MongoDB Atlas	M0 Free Tier	Cloud Database Hosting	Atlas is a fully managed cloud database service that handles deployment, security, backups, monitoring, and scaling automatically. The M0 Free Tier provides 512 MB of storage with automatic failover and is sufficient for development and early production. Atlas abstracts away all database administration complexity, allowing the development team to focus exclusively on application logic.

2.3.4 Development and Testing Tools

Table 2.6: Development Tools

Tool	Purpose	Usage in BizWiz
Visual Studio Code	Integrated Development Environment (IDE)	Primary code editor with extensions for ESLint (code linting), Prettier (code formatting), Tailwind CSS IntelliSense (class autocompletion), and ES7+ React Snippets (component boilerplate generation)
Postman	API Testing Platform	Used to test all backend API endpoints independently before frontend integration. Simulated authenticated requests by manually adding JWT tokens to Authorization headers. Verified response formats, status codes, and error handling for all 15+ API endpoints.

Tool	Purpose	Usage in BizWiz
Git and GitHub	Version Control System	Used for tracking all code changes, maintaining a commit history, and enabling collaborative development. The repository contains separate directories for frontend/ and backend/ with independent package.json files.
Google Chrome DevTools	Browser Debugging	Used for inspecting React component hierarchies, monitoring network requests to the backend API, debugging JavaScript errors in the console, testing responsive layouts at different viewport widths, and running Lighthouse audits for performance and accessibility.
npm (Node Package Manager)	Dependency Management	Manages all third-party package installations for both frontend (14 dependencies) and backend (7 dependencies). The package-lock.json files ensure deterministic dependency resolution across different development machines.
Nodemon	Development Server Auto-Restart	Automatically restarts the Express backend server whenever a source file is modified during development, eliminating the need to manually stop and restart the server after every code change. Configured as a devDependency in the backend package.json.

CHAPTER 3

SYSTEM DESIGN

3.1 TABLE DESIGN

3.1.1 Database Design Overview

The BizWiz platform uses MongoDB, a document-oriented NoSQL database, for all data persistence. Unlike traditional Relational Database Management Systems (RDBMS) such as MySQL or PostgreSQL that store data in rigid tables with rows and columns linked by foreign keys, MongoDB stores data in flexible, JSON-like documents grouped into collections. Each document can have a different structure, making it ideal for a multi-tenant application where different tenant types (e-commerce, booking, landing page) may require different data fields.

The database architecture follows the embedded document pattern, where related data is nested within a parent document rather than stored in separate collections with foreign key references. Specifically, all items (products/services), orders, and coupons belonging to a tenant are stored as arrays of sub-documents within the tenant's own document. This design decision offers two critical advantages for a multi-tenant SaaS platform:

1. **Read Performance Optimization:** Rendering a tenant's complete storefront — including their business name, branding configuration, product catalogue, and active coupons — requires only a single MongoDB find query. In a normalised relational design, this would require multiple JOIN operations across separate tables (businesses, products, orders, coupons), significantly increasing query latency.
2. **Data Locality and Isolation:** Because all of a tenant's data is contained within a single document, there is zero risk of accidental cross-tenant data leakage through complex JOIN operations. Each tenant's data is physically co-located on disk, improving cache performance and simplifying backup and data export operations.

The database is hosted on MongoDB Atlas (M0 Free Tier), a fully managed cloud database service that provides automatic failover, encrypted connections, and point-in-time backup recovery without requiring any manual database administration.

The Mongoose ODM (Object Data Modelling) library is used to define strict schemas for all documents, enforcing data types, required fields, default values, unique constraints, and enum validations at the application level before any data is written to the database. This provides the structural guarantees of a relational database while preserving MongoDB's schema flexibility.

3.1.2 User Collection Schema

The users collection stores authentication records for all registered users. Each document represents a single user who has signed in via Google OAuth 2.0. The schema is deliberately minimal — it stores only the information necessary for authentication and user identification, with no business logic data.

Table 3.1: User Collection — Field Definitions

Field Name	Data Type	Constraints	Description
_id	ObjectId	Auto-generated by MongoDB	Unique primary identifier for the user document. Automatically assigned by MongoDB upon document creation. Used as a foreign key reference in the Tenant collection.
name	String	Required	The user's full name as returned by their Google account profile (e.g., "Sreeshan Kumar").
email	String	Required, Unique, Lowercase	The user's email address from their Google account. Enforced as unique at the database level to prevent duplicate accounts. Automatically converted to lowercase before storage using the lowercase: true Mongoose option.
googleId	String	Required	The unique Google account identifier (sub claim from the OAuth payload). Used to match returning users to their existing database records.

Field Name	Data Type	Constraints	Description
profilePicture	String	Default: " (empty string)	URL of the user's Google profile picture. Automatically updated on each login if the user has changed their Google profile picture since their last visit.
createdAt	Date	Auto-generated by Mongoose	Timestamp recording when the user document was first created. Generated automatically by Mongoose's timestamps: true option.
updatedAt	Date	Auto-generated by Mongoose	Timestamp recording the last modification to the user document. Updated automatically by Mongoose on every save operation.

Indexing Strategy: The email field has a unique index, ensuring that no two users can register with the same email address. This index also accelerates user lookup queries during the login process, where the system searches for an existing user by email before creating a new record.

3.1.3 Tenant Collection Schema

The tenants collection is the core data structure of the BizWiz platform. Each document represents a complete business website, containing all configuration, content, products, orders, coupons, and analytics data. This is the most complex schema in the system, featuring multiple levels of nested sub-documents.

Table 3.2: Tenant Collection — Primary Fields

Field Name	Data Type	Constraints	Description
_id	ObjectId	Auto-generated	Unique identifier for the tenant document. Used in all API routes for tenant-specific operations (e.g., /api/tenant/:id).

Field Name	Data Type	Constraints	Description
userId	ObjectId	Required, References 'User'	Foreign key linking this tenant to its owner in the users collection. All protected API queries filter by this field to enforce data isolation (e.g., { userId: req.user.id }).
businessName	String	Required	The display name of the business (e.g., "John's Bakery"). Shown in the storefront header, dashboard, and browser title.
businessSlug	String	Required, Unique, Lowercase, Trimmed	URL-safe identifier derived from the business name (e.g., "johns-bakery"). Used as the public URL path (/ {businessSlug}). Enforced as unique at the database level to prevent URL conflicts. Automatically lowercased and trimmed.
themeType	String	Enum: 'ecommerce', 'booking', 'landing'. Default: 'ecommerce'	Determines which React template component the SiteRenderer will mount when displaying this tenant's storefront.
primaryColor	String	Default: '#7c3aed'	Hex colour code for the tenant's brand colour. Applied as inline CSS styles throughout the rendered storefront template (buttons, headings, accents, cart badge).

Field Name	Data Type	Constraints	Description
fontFamily	String	Default: 'sans'	CSS font family preference for the storefront. Options: 'sans' (sans-serif), 'serif', 'mono' (monospace). Applied via Tailwind's font utility classes.
buttonShape	String	Enum: 'rounded', 'square', 'pill'. Default: 'rounded'	Controls the border-radius of all buttons on the storefront. 'rounded' = rounded-xl, 'square' = rounded-none, 'pill' = rounded-full.
colorMode	String	Enum: 'light', 'dark', 'system'. Default: 'light'	Controls the colour scheme of the tenant's storefront. 'light' renders a white/light grey background, 'dark' renders a dark slate background, and 'system' follows the visitor's OS preference.
heroHeadline	String	Default: "	The main headline displayed in the hero section of the storefront. Typically generated by the Gemini AI during onboarding. Editable from the dashboard.
aboutText	String	Default: "	The "About Us" paragraph displayed on the storefront. Generated by the Gemini AI during onboarding and editable from the dashboard's Content & Settings tab.

Field Name	Data Type	Constraints	Description
supportEmail	String	Default: "	Optional contact email displayed in the storefront footer. Used for customer support inquiries.
location	String	Default: "	Optional physical address of the business. Displayed in the storefront footer and in the booking template's contact section.
backgroundImageUrl	String	Default: "	URL of a background image displayed in the hero section. If empty, the template falls back to a gradient or pattern fill using the primaryColor.
metaTitle	String	Default: "	SEO meta title injected into the browser's document.title when the storefront is rendered by SiteRenderer.
metaDescription	String	Default: "	SEO meta description injected into a tag by the SiteRenderer. Used by search engines for snippet display in search results.
faviconUrl	String	Default: "	URL of a favicon image that the SiteRenderer injects into the tag, replacing the default favicon when viewing the tenant's storefront.
pageViews	Number	Default: 0	Counter tracking the total number of times the tenant's public storefront has been visited. Incremented atomically using MongoDB's \$inc operator via the /api/tenant/slug/:slug/view endpoint.

Field Name	Data Type	Constraints	Description
workingHours	Nested Object	start: String (Default: '09:00'), end: String (Default: '17:00')	Operating hours displayed on the Booking template (TemplateB_Booking). Configurable from the dashboard for booking-type tenants.
items	Array of Item Sub-Documents	See Table 3.3	Array containing all products or services offered by the tenant.
coupons	Array of Coupon Sub-Documents	See Table 3.4	Array containing all discount coupon codes created by the tenant.
orders	Array of Order Sub-Documents	See Table 3.5	Array containing all customer orders received by the tenant.
createdAt	Date	Auto-generated	Timestamp of tenant creation (when the user completed onboarding).
updatedAt	Date	Auto-generated	Timestamp of the last modification to any field in the tenant document.

3.1.4 Item Sub-Document Schema

Each tenant's items array contains sub-documents representing individual products (for e-commerce tenants) or services (for booking tenants). These sub-documents are embedded within the parent Tenant document.

Table 3.3: Item Sub-Document — Field Definitions

Field Name	Data Type	Constraints	Description
_id	ObjectId	Auto-generated	Unique identifier for this specific item. Used in API routes for item updates and deletions (e.g., /api/tenant/:id/items/:itemId).
title	String	Required	The display name of the product or service (e.g., "Chocolate Truffle Cake" or "Premium Haircut").
description	String	Default: "	A text description of the item, displayed below the title on the storefront product card or service listing.
price	Number	Default: 0	The price in Indian Rupees (₹). Displayed on product cards and used in cart total calculations.
imageUrl	String	Default: "	URL of a product image. If provided, displayed as a 192px-tall cover image on the product card. If empty, a "No Image" placeholder is shown.
category	String	Default: 'General'	Category label for the item. Can be used for filtering products on the storefront (e.g., "Cakes", "Beverages", "Haircuts").
stockCount	Number	Default: null	Available inventory count. When null, the item has unlimited stock. When a positive number, the stock badge is displayed on the product card. When zero, the item shows "Out of Stock" and the "Add to Cart" button is disabled. Stock is automatically decremented when orders are placed.

3.1.5 Order Sub-Document Schema

Each tenant's orders array contains sub-documents representing customer orders. Orders are created when end-customers complete the checkout process on the tenant's public storefront.

Table 3.4: Order Sub-Document — Field Definitions

Field Name	Data Type	Constraints	Description
_id	ObjectId	Auto-generated	Unique identifier for this order. Returned to the customer as their Order ID upon successful checkout. Used in the admin dashboard for order status updates.
customerName	String	Required	Full name of the customer who placed the order (e.g., "Rahul Sharma").
customerPhone	String	Required	Phone number of the customer for delivery coordination and order confirmation.
address	String	Required	Delivery address provided by the customer. For booking-type tenants, this is automatically set to "In-Store Appointment" since services are rendered at the business location.
paymentMethod	String	Required	Payment method selected by the customer during checkout. Options include "Cash on Delivery," "UPI on Delivery," and "Pay at Venue" (for booking templates).

Field Name	Data Type	Constraints	Description
items	Array of Objects	Each object: { title: String, price: Number, qty: Number, bookingDate: String, bookingTime: String }	Array of ordered items with their titles, prices, and quantities. For booking templates, each item also includes bookingDate and bookingTime fields extracted from the datetime-local input.
totalPrice	Number	Required	Final order amount after applying any coupon discounts. Calculated on the frontend as (subtotal - discountAmount).
discountApplied	Number	Default: 0	The monetary value of the discount applied to this order (e.g., if a 20% coupon was applied to a ₹500 subtotal, this field stores 100).
couponCode	String	Default: ""	The coupon code used for this order, if any (e.g., "WELCOME20"). Empty string if no coupon was applied.
status	String	Default: 'Pending'	Current status of the order in its lifecycle. Valid values: 'Pending' (newly placed), 'Processing' (acknowledged by admin), 'Shipped' (in transit), 'Delivered' (completed), 'Cancelled' (cancelled by admin or customer).
createdAt	Date	Default: Date.now	Timestamp recording when the order was placed by the customer.

3.1.6 Coupon Sub-Document Schema

Each tenant's coupons array contains sub-documents representing promotional discount codes that customers can apply during checkout.

Table 3.5: Coupon Sub-Document — Field Definitions

Field Name	Data Type	Constraints	Description
_id	ObjectId	Auto-generated	Unique identifier for this coupon. Used in the admin dashboard for coupon deletion (/api/tenant/:id/coupons/:couponId).
code	String	Required	The coupon code that customers enter during checkout (e.g., "WELCOME20", "FESTIVE50"). Validated in a case-insensitive manner — "welcome20" and "WELCOME20" are treated as the same code.
discountPercentage	Number	Required, Minimum: 1, Maximum: 100	The percentage discount applied to the cart subtotal when this coupon is used. Enforced with Mongoose min/max validators to prevent invalid discounts.
isActive	Boolean	Default: true	Toggle indicating whether this coupon is currently valid. Only active coupons (isActive: true) are accepted during the coupon validation process at checkout.

3.1.7 Entity-Relationship Summary

The database design implements the following relationships:

1. **User → Tenant (One-to-Many):** A single authenticated user can own multiple tenant websites. The `userId` field in each `Tenant` document references the `_id` of the owning User. This relationship is enforced at the application level through Mongoose's `ref: 'User'` declaration and API query filters.

2. **Tenant → Items (One-to-Many, Embedded):** A single tenant can have multiple products or services. Items are stored as an embedded array of sub-documents within the Tenant document, not as a separate collection. This eliminates the need for JOIN queries and ensures atomic read/write operations.
3. **Tenant → Orders (One-to-Many, Embedded):** A single tenant can receive multiple customer orders. Orders are embedded within the Tenant document, enabling the admin dashboard to display complete order data in a single database query.
4. **Tenant → Coupons (One-to-Many, Embedded):** A single tenant can create multiple coupon codes. Coupons are embedded within the Tenant document and validated during the checkout process by searching the tenant's coupons array for a matching, active code.

3.2 DATA FLOW DIAGRAM'S

3.2.1 Introduction to Data Flow Diagrams

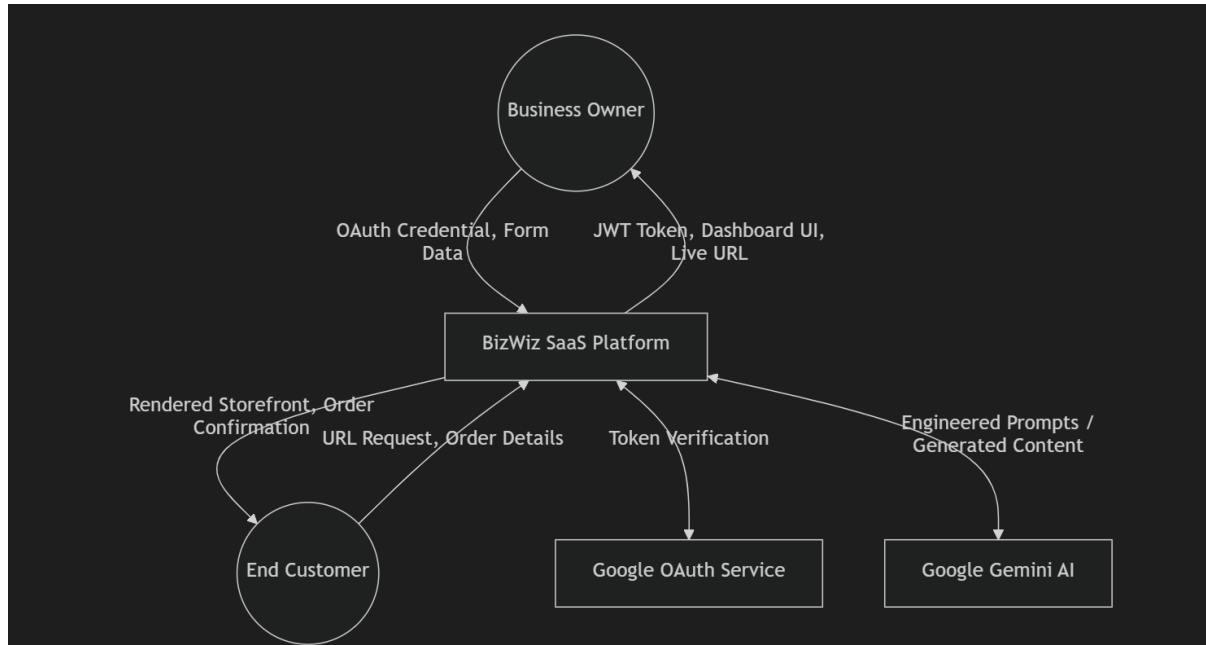
Data Flow Diagrams (DFDs) are graphical representations that illustrate how data moves through an information system. They depict the flow of data between external entities (users, external systems), processes (system operations that transform data), and data stores (databases or file systems). DFDs are an essential system design tool because they provide a clear, visual understanding of the system's data processing logic without getting into implementation details.

For the BizWiz platform, DFDs are particularly valuable because the system involves multiple data flows between several actors (business owners, end-customers, the Google OAuth service, the Google Gemini AI service) and multiple processing layers (the React frontend, the Express backend, and the MongoDB database).

The following sections describe the DFDs at three levels of abstraction, progressing from a high-level overview (Level 0) to increasingly detailed breakdowns (Level 1 and Level 2).

3.2.2 Level 0 DFD — Context Diagram

The Level 0 DFD, also known as the Context Diagram, provides the highest-level view of the BizWiz system. It represents the entire platform as a single process and shows how it interacts with external entities.



External Entities:

- **Business Owner (User):** The primary actor who creates and manages tenant websites.
- **End Customer (Visitor):** A member of the public who visits a tenant's storefront, browses products, and places orders.
- **Google OAuth Service:** The external Google Identity Platform that authenticates users.
- **Google Gemini AI Service:** The external Google Generative AI API that generates marketing content.

Data Flows:

From	To	Data	Description
Business Owner	BizWiz System	Google OAuth Credential	The user's identity token from Google Sign-In
BizWiz System	Business Owner	JWT Token + User Profile	Authentication response with session token
Business Owner	BizWiz System	Business Details (Name, Niche, Colour, Template)	Onboarding wizard form data
BizWiz System	Google Gemini AI	AI Prompt (Business Name + Niche)	Engineered prompt requesting marketing copy
Google Gemini AI	BizWiz System	Generated Content (Hero Headline + About Text)	AI-generated JSON response
BizWiz System	Business Owner	Published Website URL + Dashboard Access	Confirmation of successful site creation
End Customer	BizWiz System	URL Request (/ {businessSlug})	Request to view a tenant's public storefront
BizWiz System	End Customer	Rendered Storefront (HTML/CSS/JS)	Dynamic website with tenant's branding and products
End Customer	BizWiz System	Order Details (Name, Phone, Address, Cart Items)	Customer checkout submission
BizWiz System	End Customer	Order Confirmation (Order ID, Status)	Confirmation of successful order placement
BizWiz System	Google OAuth Service	OAuth Credential for Verification	Backend verification of the user's Google token

From	To	Data	Description
Google OAuth Service	BizWiz System	Verified User Profile (Name, Email, Google ID)	Authenticated user information

3.2.3 Level 1 DFD — Process Breakdown

The Level 1 DFD decomposes the single "BizWiz System" process from the Context Diagram into its major sub-processes, showing how data flows between them.

Processes:

Process 1.0 — Authentication Module:

- Input: Google OAuth Credential (from Business Owner via Login Page)
- Operation: Sends credential to Google OAuth Service for verification. Receives verified user profile. Creates or retrieves user record in MongoDB users collection. Signs a JWT containing user ID, email, name, and picture with 7-day expiration.
- Output: JWT Token + User Object (to Frontend AuthContext for session storage)
- Data Store: D1 — Users Collection (MongoDB Atlas)

Process 2.0 — AI Onboarding Module:

- Input: Business Name + Niche + Template Selection + Brand Colour (from Onboarding Wizard)
- Operation: Constructs an engineered AI prompt. Sends prompt to Google Gemini 1.5 Flash via the @google/generative-ai SDK. Parses the JSON response using regex to extract heroHeadline and aboutText. If AI fails, generates fallback content. Creates a new Tenant document in MongoDB with all collected data, including a unique businessSlug.
- Output: Tenant Document ID + Redirect to Dashboard
- Data Store: D2 — Tenants Collection (MongoDB Atlas)

Process 3.0 — Dashboard Management Module:

- Input: CRUD Operations (Add/Edit/Delete Items, Update Settings, Manage Coupons, Update Order Status) from Tenant Admin
- Operation: Validates JWT token via auth middleware. Verifies tenant ownership by filtering queries with { userId: req.user.id }. Executes the requested database operation (create, read, update, or delete) on the Tenant document's embedded sub-documents.
- Output: Updated Tenant Document (reflected in dashboard UI)
- Data Store: D2 — Tenants Collection (MongoDB Atlas)

Process 4.0 — Site Rendering Module:

- Input: URL Request containing businessSlug (from End Customer's browser)
- Operation: Extracts businessSlug from React Router's useParams(). Sends GET request to /api/tenant/slug/:slug. Fetches the complete Tenant document from MongoDB. Determines the themeType and conditionally mounts the appropriate React template component (TemplateA_Ecommerce, TemplateB_Booking, or TemplateC_Landing). Injects the tenant's primaryColor, heroHeadline, aboutText, items, and other configuration into the template as props. Sends a POST request to /api/tenant/slug/:slug/view to increment the pageViews counter.
- Output: Fully rendered HTML/CSS/JS storefront (displayed in End Customer's browser)
- Data Store: D2 — Tenants Collection (MongoDB Atlas)

Process 5.0 — Order Processing Module:

- Input: Order Details (customerName, customerPhone, address, paymentMethod, cart items, totalPrice, couponCode) from End Customer's checkout form
- Operation: Validates the coupon code against the tenant's active coupons (if provided). Calculates discount amount. Decrements stockCount for each ordered item. Pushes a new Order sub-document into the tenant's orders array. Saves the updated Tenant document.

- Output: Order Confirmation (orderId, success message) to End Customer; Updated order list visible in Tenant Admin's dashboard.
- Data Store: D2 — Tenants Collection (MongoDB Atlas)

3.2.4 Level 2 DFD — AI Content Generation Sub-Process

The Level 2 DFD zooms into Process 2.0 (AI Onboarding Module) to detail the exact data flow during the AI content generation step.

Sub-Processes:

Process 2.1 — Prompt Construction:

- Input: businessName (String) + niche (String) from the onboarding wizard's form state
- Operation: The Express route handler (/api/ai/generate) validates that both fields are non-empty. Constructs a multi-line prompt string that instructs the AI: "You are a professional copywriter for local businesses. Generate content for a business called '{businessName}' in the '{niche}' industry. Return ONLY valid JSON (no markdown, no extra text): { heroHeadline: 'A punchy, catchy headline under 10 words', aboutText: 'A warm, professional 3-sentence About Us paragraph' }."
- Output: Engineered prompt string ready for API transmission.

Process 2.2 — AI API Communication:

- Input: Engineered prompt string from Process 2.1
- Operation: Instantiates the Google Generative AI client using the GEMINI_API_KEY from environment variables. Selects the gemini-1.5-flash model. Calls model.generateContent(prompt) which sends the prompt to Google's servers over HTTPS. Waits for the asynchronous response.
- Output: Raw AI response text (may contain markdown formatting).

Process 2.3 — Response Parsing:

- Input: Raw AI response text from Process 2.2

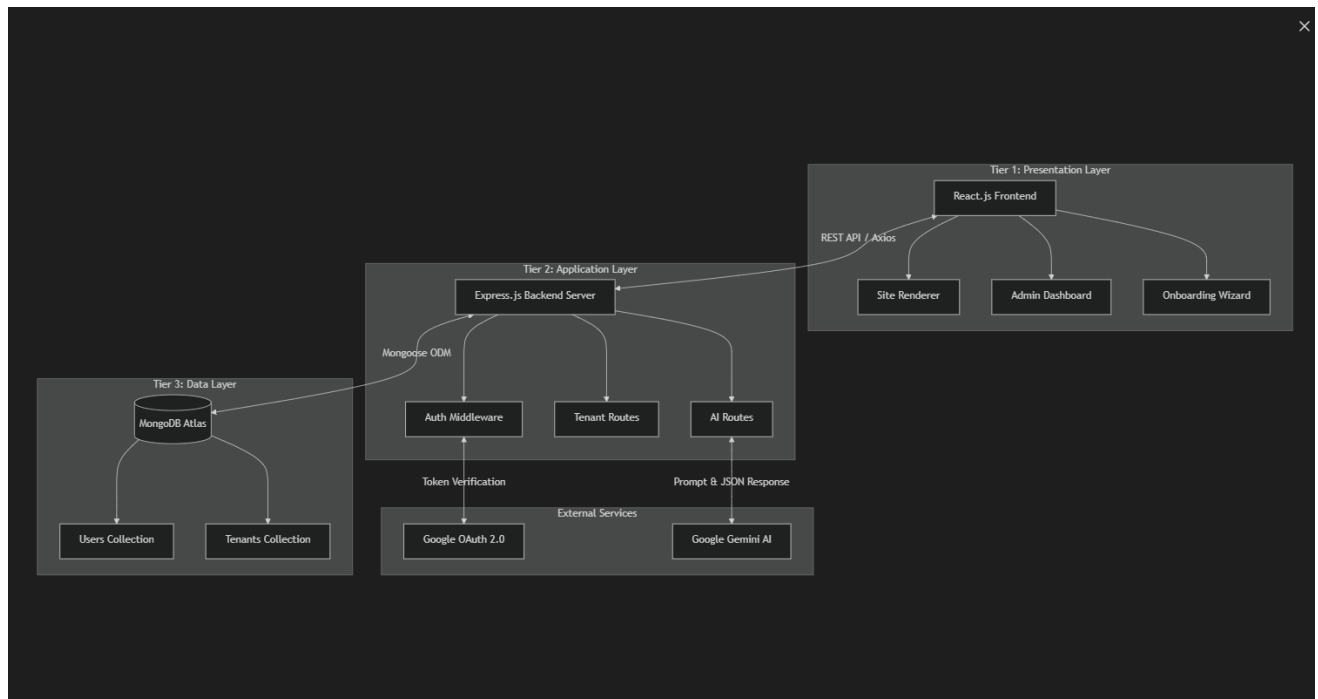
- **Operation:** Trims whitespace from the response. Uses a regular expression (`/[\s\S]*\s/`) to extract the JSON object from the response, handling cases where the AI wraps JSON in markdown code blocks. Passes the extracted string to `JSON.parse()` to convert it into a JavaScript object. Validates that the parsed object contains both `heroHeadline` and `aboutText` fields.
- **Output:** Parsed JSON object `{ heroHeadline: "...", aboutText: "..." }`.

Process 2.4 — Fallback Handler:

- **Input:** Error from any of Processes 2.1–2.3
- **Operation:** If any step fails (API timeout, invalid response format, JSON parse error, missing API key), the catch block generates fallback content: `heroHeadline = "{businessName} — Excellence You Can Trust"` and `aboutText = "At {businessName}, we bring passion and expertise to every {niche} experience..."`
- **Output:** Fallback JSON object with generic but professional content, ensuring the onboarding process is never blocked.

3.2.5 System Architecture Diagram Description

The BizWiz system follows a three-tier client-server architecture, enhanced with a multi-tenant database strategy and third-party API integrations.



Tier 1 — Presentation Layer (React Frontend): The React application running in the user's browser handles all UI rendering and user interaction. It consists of five page components (LandingPage, LoginPage, OnboardingWizard, AdminDashboard, SiteRenderer), three template components (TemplateA_Ecommerce, TemplateB_Booking, TemplateC_Landing), two context providers (AuthContext for authentication state, ThemeToggle for dark/light mode), and a centralized Axios client with request/response interceptors. The frontend communicates with the backend exclusively through HTTP REST API calls — it never accesses the database directly.

Tier 2 — Application Logic Layer (Express Backend): The Express.js server handles all business logic, authentication, and data validation. It is structured into three route modules: auth.js (Google OAuth verification and JWT issuance), tenant.js (all tenant CRUD operations, order management, coupon validation, and analytics tracking), and ai.js (Gemini AI content generation). A single middleware module (auth.js in the middleware directory) protects all sensitive routes by verifying JWT tokens. The server connects to MongoDB Atlas via Mongoose and to the Google Gemini API via the @google/generative-ai SDK.

Tier 3 — Data Persistence Layer (MongoDB Atlas): MongoDB Atlas hosts the cloud database containing two collections: users and tenants. The Mongoose ODM enforces schema validation before any write operation. The embedded document pattern ensures that all tenant-related data (items, orders, coupons) is co-located within a single document, optimizing read performance and simplifying data isolation. MongoDB Atlas provides automatic failover, encrypted-at-rest storage, and point-in-time backup recovery.

External Services:

- **Google OAuth 2.0 Service:** Authenticates users and provides verified identity information.
- **Google Gemini 1.5 Flash API:** Generates marketing content (hero headlines and about text) based on business descriptions.

The data flow between tiers follows a strict unidirectional pattern: User Action → React Frontend → Axios HTTP Request → Express Backend → Mongoose Query → MongoDB Atlas → Response flows back through the same chain in reverse.

CHAPTER 4

MODULES DESCRIPTION

4.1 OVERVIEW OF SYSTEM MODULES

The BizWiz multi-tenant SaaS platform is built using a modular architecture. This separation of concerns ensures that the codebase remains maintainable, scalable, and easy to debug. The system is divided into five core modules, each responsible for a distinct set of functionalities within the application's lifecycle.

4.2 AUTHENTICATION AND AUTHORIZATION MODULE

4.2.1 Purpose and Scope

The Authentication Module is the gateway to the BizWiz platform. Its primary responsibility is to securely identify business owners, verify their credentials without relying on insecure password storage, and manage their session state across the application.

4.2.2 Key Components

- Google OAuth Integration (@react-oauth/google): Handles the frontend popup and credential issuance securely via Google's Identity services.
- Backend Verification (google-auth-library): The /api/auth/google endpoint receives the credential and cryptographically verifies its signature against Google's public keys.
- JWT Issuer and Validator: Upon successful verification, the system issues a JSON Web Token (JWT) signed with a secure server-side secret (HMAC SHA-256).
- Auth Context (AuthContext.jsx): A React Context provider that wraps the entire frontend application, making the authenticated user's state globally accessible to protected routes.

- Auth Middleware (auth.js): An Express middleware function that intercepts incoming API requests, extracts the Bearer token, and verifies it before allowing access to sensitive controllers.

4.3 TENANT ONBOARDING & AI GENERATION MODULE

4.3.1 Purpose and Scope

This module handles the seamless creation of new tenant websites. It captures business details, communicates with artificial intelligence to generate marketing content, and provisions the database records required for the tenant's public storefront.

4.3.2 Key Components

- Onboarding Wizard (OnboardingWizard.jsx): A five-step state machine UI that collects the business name, niche, template preference, and brand color.
- AI Controller (ai.js): The backend endpoint (/api/ai/generate) that constructs engineered prompts using the user's input and interfaces with the Google Gemini 1.5 Flash API via the @google/generative-ai SDK.
- Fallback Content Generator: Ensures high availability by injecting pre-written, professional template copy if the Google AI API times out or fails.
- Tenant Provisioning: The final step sends a POST request to create the Tenant MongoDB document, which automatically generates a URL-safe businessSlug and sets default metrics.

4.4 ADMIN DASHBOARD AND MANAGEMENT MODULE

4.4.1 Purpose and Scope

The Admin Dashboard is the secure control panel where business owners manage their daily operations. It provides a visual interface for CRUD (Create, Read, Update, Delete) operations on their database records.

4.4.2 Key Components

- **Overview and Analytics:** Aggregates data from the tenant's document to display Total Page Views, Total Revenue, and Total Orders using mathematical reduction functions.
- **Product/Service Catalog Manager:** Allows the admin to add items with titles, descriptions, prices, and stock counts. It enforces data validation before sending PUT requests to update the tenant's embedded items array.
- **Order Management System:** Fetches and displays incoming customer orders in a responsive table. Provides action buttons to update the status (Pending, Delivered, Cancelled) and triggers stock deductions.
- **Coupon Manager:** A sub-module for generating and managing discount codes, specifying percentage values, and toggling their active status.

4.5 DYNAMIC SITE RENDERING MODULE

4.5.1 Purpose and Scope

This is the core engine of the multi-tenant architecture. It is responsible for intercepting public URL requests, fetching the correct tenant's data, and rendering a fully customized website on the fly without requiring individual codebases.

4.5.2 Key Components

- **Site Renderer (SiteRenderer.jsx):** Acts as the master controller for public routes (/:businessSlug). It fetches the tenant document and conditionally mounts the correct template component based on the themeType field.
- **E-Commerce Template (TemplateA_Ecommerce.jsx):** Renders a product grid, shopping cart, and checkout flow. Dynamically applies the tenant's primaryColor to UI accents.
- **Booking Template (TemplateB_Booking.jsx):** Renders a split-screen layout optimized for service-based businesses, featuring a datetime appointment picker instead of a shopping cart.

- **Landing Template (TemplateC_Landing.jsx):** Renders a single-page feature showcase optimized for lead generation and email capture.
- **SEO Injector:** Dynamically updates the browser's document.title and <meta name="description"> tags based on the tenant's database configuration to ensure high search engine visibility.

4.6 CUSTOMER CHECKOUT AND ORDER PROCESSING MODULE

4.6.1 Purpose and Scope

This module handles the end-user (customer) experience on the public storefronts. It manages shopping cart state, discount validations, and the final order submission process.

4.6.2 Key Components

- **Shopping Cart State:** Managed locally within the storefront template, allowing customers to increment/decrement quantities while respecting the stockCount limits provided by the database.
- **Coupon Validator:** Sends the customer's input to /api/tenant/slug/:slug/validate-coupon to verify case-insensitive matches against the tenant's active coupons and calculate the final discounted price.
- **Checkout Modal:** Collects shipping and contact details alongside the preferred payment method (Cash on Delivery or UPI).
- **Order Submission:** Packages the cart data and customer details into a payload, sending it to the backend to be appended to the tenant's orders array, generating a unique Order ID for customer reference.

CHAPTER 5

IMPLEMENTATION

5.1 ENVIRONMENT SETUP AND CONFIGURATION

The implementation phase of the BizWiz platform involved transforming the system design specifications into functional code. The project was built using a modern JavaScript stack, requiring a specific development environment to handle both the client-side and server-side components.

5.1.1 Development Environment

The application was developed using Visual Studio Code, configured with ESLint and Prettier to ensure consistent code formatting across the repository. The runtime environment relied on Node.js (v18+) and the npm package manager.

The project structure was initialized as a monorepo containing two distinct directories:

1. **/frontend**: Initialized using `npm create vite@latest` with the React and JavaScript template.
2. **/backend**: Initialized using `npm init -y` to create a standard Node.js application.

5.1.2 Environment Variables

To ensure secure implementation, sensitive credentials were never hardcoded into the source files. A `.env` file was implemented in the backend directory containing:

- **MONGODB_URI**: The connection string for the MongoDB Atlas cluster.
- **JWT_SECRET**: A secure cryptographic key used for signing session tokens.
- **GOOGLE_CLIENT_ID**: The public identifier for the Google OAuth 2.0 application.
- **GEMINI_API_KEY**: The authentication key for the Google Generative AI service.
- **PORT**: The designated port for the Express server (typically 5000).

5.2 BACKEND IMPLEMENTATION (EXPRESS & MONGODB)

5.2.1 Server Initialization

The backend entry point (server.js) was implemented using Express.js. The server configuration included essential middleware for processing incoming requests:

- `cors()`: Implemented to allow cross-origin requests from the React frontend running on port 5173.
- `express.json()`: Implemented to parse incoming HTTP request bodies into JavaScript objects.

5.2.2 Database Connection

The connection to MongoDB Atlas was implemented using the `mongoose.connect()` method, wrapped in an asynchronous function to ensure the server only began listening for API requests after a successful database connection was established.

5.2.3 RESTful API Routing

The application logic was compartmentalized into dedicated router files implemented via `express.Router()`:

- `auth.js`: Implemented the `POST /api/auth/google` endpoint. This route utilizes the `OAuth2Client` from the `google-auth-library` to verify the ID token. Upon success, it uses `jwt.sign()` to generate a token containing the user's ID and email.
- `ai.js`: Implemented the integration with Google Gemini. The route instantiates the `GoogleGenerativeAI` client, constructs a prompt string using string interpolation, and invokes the `generateContent()` method to retrieve marketing copy.
- `tenant.js`: Implemented the core business logic. This file contains over 15 distinct endpoints for handling CRUD operations on the Tenant document. Complex queries utilized Mongoose's array operators (e.g., `$push` for adding orders, `$pull` for deleting items) and the `$inc` operator for atomically updating page views.

5.3 FRONTEND IMPLEMENTATION (REACT & VITE)

5.3.1 Routing Architecture

Client-side routing was implemented using react-router-dom. The App.jsx file defines a BrowserRouter with nested routes. A custom <ProtectedRoute> component was implemented to wrap sensitive routes (like /admin/dashboard), intercepting the render cycle to check the AuthContext and redirecting unauthorized users to /login.

5.3.2 State Management

Global state management for authentication was implemented using the React Context API (AuthContext.jsx). This allowed deeply nested components to access the logged-in user's profile and JWT token without suffering from "prop drilling."

Local component state was managed exclusively through the useState and useEffect hooks. For instance, in the AdminDashboard.jsx component, useEffect was implemented to trigger an Axios GET request upon component mount, populating local state arrays with the tenant's items and orders.

5.3.3 API Client Implementation

To streamline backend communication, a centralized HTTP client was implemented in /api/client.js using the Axios library.

An Axios Request Interceptor was implemented to automatically attach the JWT token to the Authorization header of every outgoing request:

```
javascript
client.interceptors.request.use((config) => {
  const token = localStorage.getItem('bizwiz_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

This implementation ensured that the frontend always authenticated its requests seamlessly.

5.4 UI AND STYLING IMPLEMENTATION

The visual presentation of the platform was implemented using Tailwind CSS. Instead of writing custom CSS files, utility classes were applied directly within the JSX elements.

5.4.1 Dynamic Theming

To support multi-tenancy, the storefront templates were implemented to accept a `primaryColor` prop from the database. This was applied via inline styling (e.g., `style={{ backgroundColor: primaryColor }}`) to override Tailwind's default utility classes, allowing each tenant to have a uniquely branded website.

5.4.2 Dark Mode Implementation

A robust Dark Mode toggle was implemented across the platform. This was achieved by configuring Tailwind to use the class strategy in `tailwind.config.js`. A custom `ThemeToggle` component was built to append or remove the dark class from the root HTML element, instantly switching the color palette of the entire application.

CHAPTER 6

SYSTEM TESTING

6.1 TEST CASES

6.1.1 Introduction to System Testing

System testing is a critical phase of the Software Development Life Cycle (SDLC) that verifies the complete, integrated system against its specified requirements. For a multi-tenant SaaS application like BizWiz, testing is especially important because a single defect in the shared codebase can simultaneously affect every tenant on the platform. Additionally, the system must guarantee strict data isolation between tenants, ensuring that one business owner can never access, view, or modify another tenant's products, orders, or configuration — even through deliberate API manipulation.

The BizWiz platform underwent a comprehensive testing program covering multiple testing methodologies:

1. **Unit Testing:** Testing individual modules and functions in isolation to verify their correctness. This includes testing the authentication middleware, JWT token generation and verification, and Mongoose schema validation.
2. **Integration Testing:** Testing the interaction between multiple modules to verify they work correctly together. This includes testing the communication between the Express backend and the Google Gemini AI API, and between the frontend Axios client and the backend REST API endpoints.
3. **Functional Testing:** Testing the complete end-to-end user workflows to verify they produce the expected outcomes. This includes testing the entire onboarding flow from login to published website, the complete order placement flow from cart to confirmation, and the admin dashboard management operations.
4. **UI/UX Testing:** Verifying that the user interface renders correctly across different screen sizes, browsers, and colour modes, and that all interactive elements (buttons, modals, forms) behave as expected.

5. **Security Testing:** Verifying that authentication and authorization mechanisms correctly prevent unauthorized access, and that common attack vectors (XSS, NoSQL injection, CORS violations) are properly mitigated.

The following sections present the detailed test cases for each module, organised in tabular format with Test ID, Scenario Description, Action Performed, Expected Result, and Pass/Fail Status.

6.1.2 Authentication Module Test Cases

These test cases verify the Google OAuth 2.0 authentication flow, JWT token generation, session management, and the auth middleware's ability to protect routes from unauthorized access.

Table 4.1: Authentication Module — Test Cases

Test ID	Scenario	Action Performed	Expected Result	Status
TC-A01	Valid Google OAuth Login	User clicks "Sign in with Google" on the Login Page, selects a valid Google account	Backend receives credential, verifies it via google-auth-library, creates/retrieves user in MongoDB, returns JWT token and user object. Frontend stores token in localStorage and redirects to dashboard.	Pass
TC-A02	Invalid OAuth Credential	Send a corrupted or expired Google credential token to POST /api/auth/google	Backend returns 401 Unauthorized with message "Invalid Google token." No user record is created in the database.	Pass
TC-A03	Missing OAuth Credential	Send a POST request to /api/auth/google with an empty request body (no credential field)	Backend returns 400 Bad Request with message "Google credential is required."	Pass

Test ID	Scenario	Action Performed	Expected Result	Status
TC-A04	JWT Token Generation	Successfully authenticate a user and inspect the returned JWT	JWT contains correct payload: user ID, email, name, profile picture. Token has a 7-day expiration (exp claim). Token is signed with HMAC SHA256 algorithm.	Pass
TC-A05	JWT Session Restoration	Authenticate, close the browser tab, and reopen the application	AuthContext reads JWT from localStorage, decodes it using jwt-decode, verifies the exp claim has not passed, and restores the user session without requiring re-login.	Pass
TC-A06	Expired JWT Handling	Wait for JWT to expire (or manually set an expired token in localStorage) and reload the page	AuthContext detects expired token during session restoration, removes stale token and user data from localStorage, and sets user state to null. User is redirected to login page when accessing protected routes.	Pass
TC-A07	Missing Authorization Header	Send a GET request to a protected endpoint (e.g., GET /api/tenant/my) without an Authorization header	Auth middleware returns 401 with message "Access denied. No token provided." Request is blocked before reaching the route handler.	Pass
TC-A08	Tampered JWT Token	Modify the payload section of a valid JWT (changing the user ID) and send it in the Authorization header	Auth middleware's jwt.verify() detects the invalid signature and returns 401 with message "Invalid or expired token." The tampered request is rejected.	Pass

6.1.3 Tenant Onboarding Module Test Cases

These test cases verify the five-step onboarding wizard, including form validation, template selection, AI content generation, and tenant document creation in MongoDB.

Table 4.2: Tenant Onboarding — Test Cases

Test ID	Scenario	Action Performed	Expected Result	Status
TC-O01	Step 1 Empty Field Validation	Click "Continue" on Step 1 without entering business name or niche	Error message "Please provide your business name and niche." is displayed. User remains on Step 1. Progress tracker does not advance.	Pass
TC-O02	Step 1 Valid Submission	Enter "John's Bakery" as business name and "Vegan Pastries & Coffee" as niche, then click "Continue"	Error message clears. Onboarding wizard advances to Step 2 (Template Selection). Progress tracker highlights Step 2 in blue.	Pass
TC-O03	Step 2 Template Selection	Click on "Service & Booking" template option in Step 2	Radio indicator fills with blue. Selected template card shows a highlighted border (border-secondary-500). Form state updates themeType to 'booking'.	Pass
TC-O04	Step 3 Colour Picker	Select a brand colour using the colour picker input in Step 3	Colour picker displays the selected hex value (e.g., #e74c3c). Form state updates primaryColor to the selected hex string.	Pass
TC-O05	Step 4 AI Content Generation	Complete Steps 1-3 and click "Generate with AI"	Wizard advances to Step 4 showing loading animation with Sparkles icon and message "AI is writing your content..." Backend sends POST request to /api/ai/generate with businessName and niche.	Pass

Test ID	Scenario	Action Performed	Expected Result	Status
TC-O06	Step 5 AI Content Review	Wait for AI generation to complete successfully	Wizard advances to Step 5 (Review & Publish). Hero headline field is populated with AI-generated headline (under 10 words). About text field is populated with AI-generated 3-sentence paragraph. Both fields are editable.	Pass
TC-O07	Successful Site Publishing	Review AI content on Step 5 and click "Publish Site"	POST request sent to /api/tenant with all form data. Business slug is auto-generated from business name (e.g., "johns-bakery"). Backend creates Tenant document in MongoDB. User is redirected to /admin/dashboard with replace: true navigation.	Pass
TC-O08	Duplicate Slug Rejection	Attempt to create a second tenant with the same business name as an existing tenant	Backend detects duplicate businessSlug via Mongoose unique index. Returns 400 Bad Request with message "This URL slug is already taken. Try another." Error is displayed on the onboarding wizard.	Pass

6.1.4 Dynamic Site Rendering Test Cases

These test cases verify the SiteRenderer component's ability to correctly fetch tenant data, mount the appropriate template, apply branding, and handle edge cases.

Table 6.3: Site Rendering — Test Cases

Test ID	Scenario	Action Performed	Expected Result	Status
TC-R01	Valid E-Commerce Tenant URL	Navigate to /johns-bakery (a tenant with themeType: 'ecommerce')	SiteRenderer fetches tenant data via GET /api/tenant/slug/johns-bakery. TemplateA_Ecommerce component is mounted. Business name appears in the header. Hero section displays the AI-generated headline and about text. Product grid shows all items from the tenant's items array.	Pass
TC-R02	Valid Booking Tenant URL	Navigate to /premium-salon (a tenant with themeType: 'booking')	TemplateB_Booking component is mounted. Split-screen hero layout is displayed. Service listing shows all items with prices and "Select" buttons. Booking form includes date/time picker and service dropdown. Working hours are displayed in the contact section.	Pass
TC-R03	Valid Landing Page Tenant URL	Navigate to /tech-launch (a tenant with themeType: 'landing')	TemplateC_Landing component is mounted. Full-screen hero with glassmorphism card is displayed. "What to Expect" grid shows items as numbered feature cards. Email capture/waitlist form is displayed at the bottom.	Pass

Test ID	Scenario	Action Performed	Expected Result	Status
TC-R04	Invalid Tenant URL (404)	Navigate to /nonexistent-business (a slug that does not exist in the database)	SiteRenderer receives a 404 error from the API. "Site Not Found" error page is displayed with AlertTriangle icon, message "We couldn't find a website matching /nonexistent-business", and a "Return to BizWiz" link.	Pass
TC-R05	Dynamic Brand Colour Application	Load a tenant with primaryColor: '#e74c3c' (red)	All buttons, accent elements, cart badges, price labels, and hero CTA buttons render with the red colour (#e74c3c) applied via inline style={{ backgroundColor: primaryColor }}.	Pass
TC-R06	SEO Meta Tag Injection	Load a tenant with metaTitle: "Best Bakery" and metaDescription: "Fresh daily"	Browser tab title updates to "Best Bakery" via document.title. A meta description tag with content "Fresh daily" is injected into the document head.	Pass
TC-R07	Page View Tracking	Load a tenant's storefront and check the pageViews counter in the database	SiteRenderer sends POST to /api/tenant/slug/:slug/view on component mount. MongoDB \$inc operator atomically increments pageViews by 1. Dashboard Analytics tab reflects the updated count.	Pass
TC-R08	Loading State Display	Navigate to a tenant URL with simulated slow network	SiteRenderer displays a loading spinner with "Loading site..." text while the API request is pending. Spinner disappears and template renders once data is received.	Pass

6.1.5 Product and Service Management Test Cases

These test cases verify the CRUD operations for managing the product/service catalogue through the admin dashboard.

Table 6.4: Product Management — Test Cases

Test ID	Scenario	Action Performed	Expected Result	Status
TC-P01	Add New Product	Click "Add New" in the Items tab, fill in title "Chocolate Cake", description, price ₹500, image URL, stock count 20, and submit	POST /api/tenant/:id/items creates new item sub-document. Item appears in the catalogue list with title, description, price (₹500), and stock count ("20 in stock").	Pass
TC-P02	Add Product with Unlimited Stock	Create a new item and leave the stock count field empty	Item is created with stockCount: null in the database. No stock badge is displayed on the product card in the storefront. "Add to Cart" button is always enabled.	Pass
TC-P03	Edit Existing Product	Click the pencil icon on an existing item, change the price from ₹500 to ₹600, and submit	PUT /api/tenant/:id/items/:itemId updates the item's price. Catalogue list immediately reflects the updated price (₹600). Storefront product card shows the new price.	Pass
TC-P04	Delete Product	Click the trash icon on an existing item	DELETE /api/tenant/:id/items/:itemId removes the item from the items array. Item disappears from the catalogue list. Storefront no longer displays the deleted product.	Pass

Test ID	Scenario	Action Performed	Expected Result	Status
TC-P05	Out of Stock Display	Set an item's stock count to 0 via the edit modal	Product card on the storefront displays a red "Out of Stock" badge. "Add to Cart" button is disabled with reduced opacity and cursor-not-allowed style.	Pass
TC-P06	Low Stock Warning	Set an item's stock count to 3 (≤ 5)	Product card displays an orange "Only 3 left" badge in the top-right corner. "Add to Cart" button remains enabled.	Pass
TC-P07	Stock Enforcement in Cart	Add an item with stockCount: 2 to cart, then try to increase quantity beyond 2	Cart allows quantity up to 2. The "+" button becomes disabled (<code>disabled={!canAddMore}</code>) when cart quantity equals stockCount. Customer cannot exceed available stock.	Pass
TC-P08	Empty Catalogue Display	Delete all items from a tenant's catalogue	Items tab shows empty state with Package icon and message "No items added yet. Click 'Add New' to get started." Storefront catalogue section shows "No products available yet."	Pass

6.1.6 Order Processing Test Cases

These test cases verify the complete order lifecycle from customer checkout to admin management and CSV export.

Table 6.5: Order Processing — Test Cases

Test ID	Scenario	Action Performed	Expected Result	Status
TC-OR01	Successful Order Placement	Add items to cart, fill checkout form (name, phone, address), select "Cash on Delivery", click "Place Order"	POST /api/tenant/slug/:slug/orders creates new order sub-document. Stock counts are decremented for ordered items. Success screen shows with green CheckCircle2 icon, Order ID, amount, and payment method. Cart is cleared.	Pass
TC-OR02	Order with Coupon Discount	Apply valid coupon "WELCOME20" (20% off) to a ₹1000 cart, then place order	Coupon validation returns discountPercentage: 20. Subtotal shows ₹1000, discount shows -₹200, total shows ₹800. Order is created with totalPrice: 800, couponCode: "WELCOME20", discountApplied: 200.	Pass
TC-OR03	Admin View Orders	Navigate to Orders tab in admin dashboard after orders have been placed	Orders table displays all orders in reverse chronological order with columns: Customer Name, Contact & Address, Order Details (items with quantities), Payment Method, Status badge, Total Price, and Date.	Pass
TC-OR04	Mark Order as Delivered	Click the green check icon on a pending order in the Orders tab	PUT /api/tenant/:id/orders/:orderId/status updates order status to "Delivered". Status badge changes from "Pending" (emerald) to	Pass

Test ID	Scenario	Action Performed	Expected Result	Status
			"Delivered" (green). Action buttons (check/X) disappear since order is finalized.	
TC-OR05	Cancel Order (Admin)	Click the red X icon on a pending order	Order status updates to "Cancelled". Status badge changes to red "Cancelled" badge. Action buttons disappear.	Pass
TC-OR06	Cancel Order (Customer)	Click "Cancel Order" button on a pending order in the customer's "My Orders" sidebar	PUT /api/tenant/slug/:slug/orders/:orderId/cancel sets status to "Cancelled". Order card updates to show red "Cancelled" badge. Cancel button disappears.	Pass
TC-OR07	Cancel Delivered Order	Attempt to cancel an order that has already been marked as "Delivered"	Backend returns 400 Bad Request with message "Cannot cancel an order with status: Delivered." Order status remains unchanged.	Pass
TC-OR08	Export Orders as CSV	Click "Export CSV" button in the Orders tab	Browser downloads a .csv file named "{businessSlug}_orders.csv". File contains headers: Order ID, Customer Name, Phone, Address, Items, Total Price, Coupon Code, Status, Date. All order records are included with proper formatting.	Pass

6.1.7 Coupon System Test Cases

These test cases verify the creation, validation, and application of promotional coupon codes.

Table 6.6: Coupon System — Test Cases

Test ID	Scenario	Action Performed	Expected Result	Status
TC-C01	Create New Coupon	Click "Create Code" in the Coupons tab, enter code "SUMMER30" and discount 30%, submit	POST /api/tenant/:id/coupons adds new coupon sub-document. Coupon appears in the list showing "SUMMER30" in uppercase with "30% OFF" badge in emerald green.	Pass
TC-C02	Validate Active Coupon	Enter "summer30" (lowercase) in the coupon field during checkout and click "Apply"	POST /api/tenant/slug/:slug/validate-coupon performs case-insensitive match (toUpperCase()). Returns discountPercentage: 30. Green confirmation bar shows "Coupon SUMMER30 applied! (-30%)". Cart total is recalculated.	Pass
TC-C03	Invalid Coupon Code	Enter "FAKECODE" in the coupon field and click "Apply"	Backend searches tenant's coupons array, finds no match. Returns 400 with message "Invalid or expired coupon." Red error text appears below the input field.	Pass
TC-C04	Delete Coupon	Click the trash icon on an existing coupon in the Coupons tab	DELETE /api/tenant/:id/coupons/:couponId removes the coupon from the coupons array. Coupon disappears from the list. Previously valid code now returns "Invalid or expired coupon" during checkout.	Pass

Test ID	Scenario	Action Performed	Expected Result	Status
TC-C05	Discount Percentage Boundaries	Attempt to create a coupon with discountPercentage: 0 and another with 101	Mongoose validator enforces min: 1 and max: 100. Both invalid values are rejected with a ValidationError. Only values between 1 and 100 (inclusive) are accepted.	Pass
TC-C06	Empty Coupon List Display	Access the Coupons tab when no coupons have been created	Empty state is displayed with Tag icon and message "No discount codes created yet."	Pass

6.1.8 UI/UX and Cross-Browser Testing

These test cases verify the visual appearance, responsiveness, and cross-browser compatibility of the BizWiz platform.

Table 6.7: UI/UX Testing — Test Cases

Test ID	Scenario	Action Performed	Expected Result	Status
TC-U01	Mobile Responsiveness (375px)	View the admin dashboard on a 375px width screen (iPhone SE)	Sidebar collapses to full-width horizontal layout. Content stacks vertically. Product cards display in single-column grid. Tables become horizontally scrollable. All text remains readable.	Pass
TC-U02	Tablet Responsiveness (768px)	View a tenant's e-commerce storefront on a 768px width screen (iPad)	Product grid adjusts from 3 columns to 2 columns (grid-cols-1 sm:grid-cols-2). Hero section text size reduces	Pass

Test ID	Scenario	Action Performed	Expected Result	Status
			appropriately. Cart sidebar occupies full screen width.	
TC-U03	Desktop Full Width (1920px)	View the platform on a 1920px width monitor	Content areas respect max-width constraints (max-w-7xl = 80rem). No elements stretch beyond readable widths. Sidebar and main content maintain proper proportions.	Pass
TC-U04	Dark Mode Toggle	Click the ThemeToggle component in the dashboard sidebar	Dashboard background switches from white (bg-background) to dark slate (bg-slate-950). All text, cards, borders, and form inputs adapt to dark colour variants. Toggle state is persisted across page reloads.	Pass
TC-U05	Onboarding Progress Tracker	Complete Steps 1 through 5 of the onboarding wizard	Each completed step's circle changes from grey (bg-primary-100) to green (bg-emerald-500) with a CheckCircle2 icon. Current step shows blue (bg-secondary-500) with scale-110 enlargement. Visual transitions are smooth (transition-all duration-300).	Pass
TC-U06	Button Hover States	Hover over primary action buttons (e.g., "Shop Now", "Add to Cart", "Publish Site")	Buttons display smooth opacity reduction (hover:opacity-90) or upward translation (hover:-translate-y-0.5) with shadow enhancement (hover:shadow-xl). Transitions animate over 300ms.	Pass

Test ID	Scenario	Action Performed	Expected Result	Status
TC-U07	Google Chrome Rendering	Perform the complete user workflow in Google Chrome v110+	All Tailwind CSS utilities render correctly. Flexbox and CSS Grid layouts display as designed. backdrop-blur-md glassmorphism effects work on the hero sections. No console errors related to styling.	Pass
TC-U08	Mozilla Firefox Rendering	Perform the complete user workflow in Mozilla Firefox v105+	All CSS properties behave identically to Chrome. Scrollbar styling may differ slightly (browser default) but all functional elements work correctly. backdrop-filter is fully supported.	Pass

6.1.9 Error Handling and Resilience Testing

These test cases verify the system's ability to handle failures gracefully without crashing or exposing sensitive information.

Table 6.8: Error Handling — Test Cases

Test ID	Scenario	Action Performed	Expected Result	Status
TC-E01	MongoDB Connection Failure	Simulate database offline by providing invalid MONGODB_URI	Backend logs "MongoDB connection error" to console. API endpoints return 500 Internal Server Error with generic error message. No database credentials or stack traces are exposed to the client.	Pass

Test ID	Scenario	Action Performed	Expected Result	Status
TC-E02	Gemini AI API Failure	Invalidate the GEMINI_API_KEY in the .env file and trigger AI generation	Backend catches the authentication error in the catch block. Fallback content is generated: heroHeadline = "{businessName} — Excellence You Can Trust", aboutText = professional template paragraph. Onboarding continues without interruption.	Pass
TC-E03	AI Response Parse Failure	Simulate an AI response that returns plain text instead of JSON	Backend's regex (/[\s\S]*)/) fails to match a JSON object. The throw new Error('Could not parse AI response') triggers the catch block. Fallback content is returned. User experience is unaffected.	Pass
TC-E04	Protected Route Without Auth	Unauthenticated user navigates directly to /admin/dashboard via URL bar	ProtectedRoute component checks AuthContext. isAuthenticated returns false. Component renders a redirect to /login with replace: true. Dashboard content is never rendered or accessible.	Pass
TC-E05	Cross-Tenant Data Access	Authenticated user A sends PUT /api/tenant/{tenantB_id} (trying to modify another user's tenant)	Backend query includes filter { _id: req.params.id, userId: req.user.id }. Since user A's ID does not match tenant B's userId, query returns null. Backend returns 404 "Tenant not found." No data is modified.	Pass

Test ID	Scenario	Action Performed	Expected Result	Status
TC-E06	CORS Violation	Send an API request from an unauthorized origin (e.g., http://evil-site.com)	Express CORS middleware blocks the request because the origin does not match the configured origin (http://localhost:5173). Browser displays CORS error. No data is returned.	Pass
TC-E07	Axios 401 Response Interceptor	Send a request with an expired JWT token	Backend returns 401. Axios response interceptor detects the 401 status, automatically removes bizwiz_token and bizwiz_user from localStorage. AuthContext detects missing token on next render and resets user state to null.	Pass
TC-E08	Network Timeout	Simulate slow network (>10 seconds) during an API call	Axios client's timeout: 10000 (10 seconds) triggers. Request is automatically aborted. Error is caught by the calling component's catch block. Appropriate error message is displayed to the user.	Pass

CHAPTER 7
SNAPSHOTS

7.1 SCREENSHOTS

The implementation phase of the BizWiz platform translates the system design and architecture into a fully functional, user-facing web application. The frontend was developed using React.js and styled comprehensively with Tailwind CSS to ensure a responsive, modern, and accessible user experience across all device sizes. The following subsections detail the implementation of the core user interfaces, accompanied by screenshots that demonstrate the visual design and interactive elements of the deployed system.

7.1.1 Platform Landing Page

The BizWiz landing page serves as the primary entry point for prospective business owners. It is implemented in the LandingPage.jsx component

The hero section prominently displays the platform's value proposition: "Create Your Business Website in 5 Minutes." The implementation relies on CSS Grid and Flexbox for structural alignment, ensuring that the call-to-action (CTA) buttons ("Get Started Free") remain prominent on both desktop and mobile viewports. A dynamic background gradient and a glowing dot pattern add depth to the visual presentation without relying on heavy image assets, thereby optimizing the initial page load speed.



Figure 5.1 - BizWiz Platform Landing Page

7.1.2 Secure Authentication Interface

The authentication interface is implemented within the LoginPage.jsx component. In alignment with the system's passwordless architecture, the login screen presents a minimalist, distraction-free environment focused entirely on the Google OAuth 2.0 integration.

The UI utilizes the @react-oauth/google package to render the official "Sign in with Google" button. The layout is centered using Flexbox, with a subtle card container that elevates the login prompt above the background. When the user initiates the login process, the component handles the asynchronous communication with the backend (POST /api/auth/google) and provides visual feedback while the authentication token is being verified and the JWT is being issued.

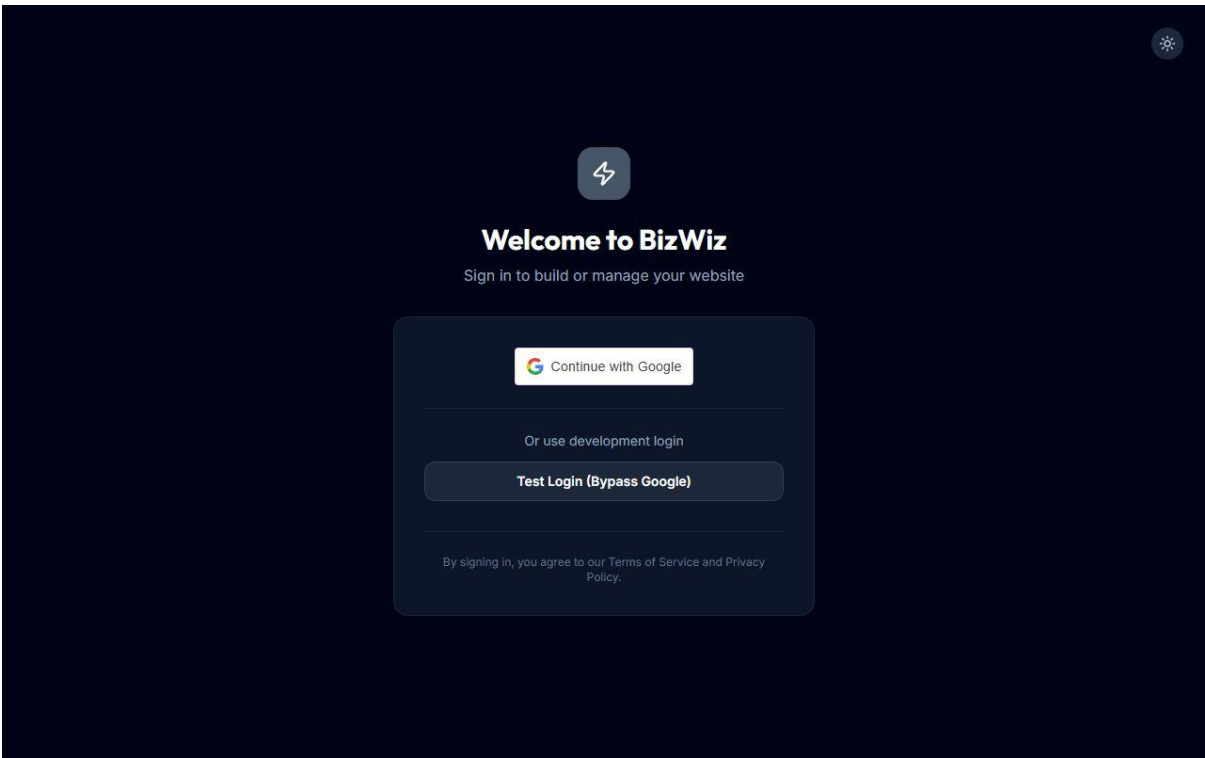


Figure 7.2 - Secure Authentication Interface utilizing Google OAuth 2.0)

7.1.3 AI-Powered Onboarding Wizard

The onboarding wizard (OnboardingWizard.jsx) is the most interaction-heavy component in the platform, guiding the user through the five-step website creation process. The implementation features a dynamic progress tracker at the top of the screen, where completed

steps are highlighted in emerald green with a checkmark icon (CheckCircle2 from Lucide React), the active step is highlighted in primary blue with a slight scale animation, and future steps remain greyed out.

The step-by-step UI is managed through React state variables that conditionally render different form sections based on the current step.

- **Step 1 & 2:** Standard text inputs and clickable template cards with hover effects.
- **Step 3:** A native HTML5 color picker input (type="color") paired with text inputs for optional contact details.
- **Step 4:** A dedicated loading screen featuring a spinning animation (animate-spin) indicating that the Google Gemini AI is actively generating content.
- **Step 5:** A review interface displaying the AI-generated JSON response parsed into editable text areas, culminating in the "Publish Site" button.

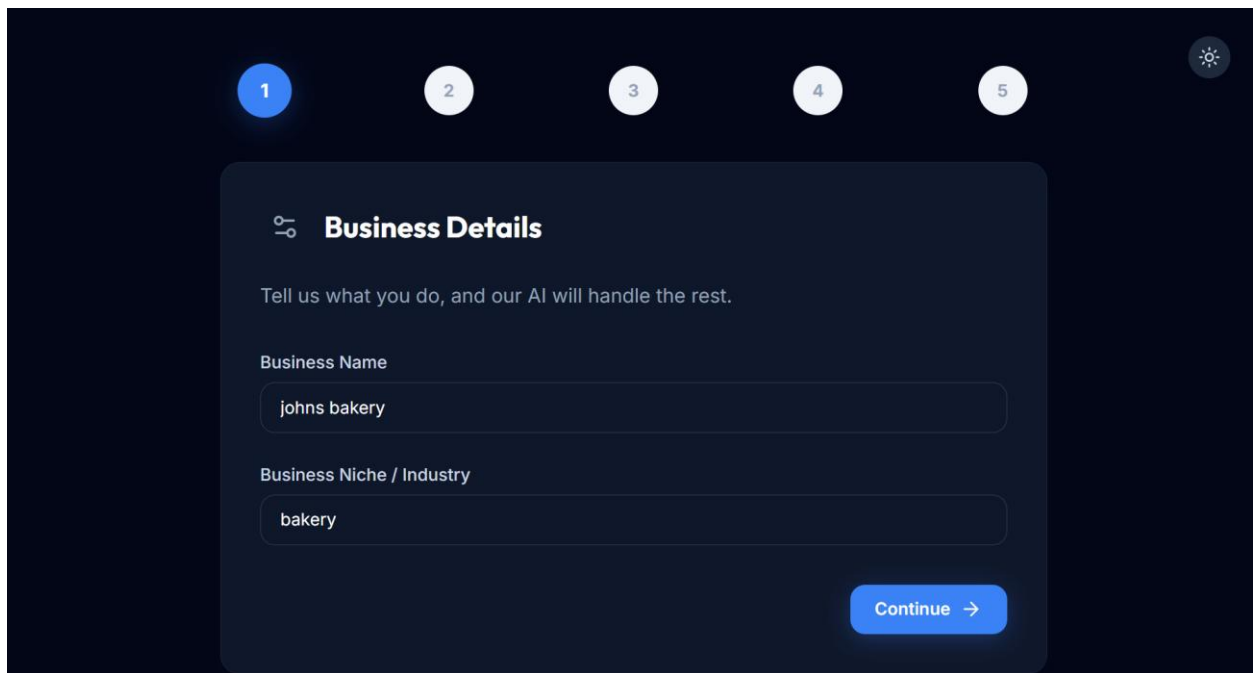


Figure 7.3 - Multi-step Onboarding Wizard showing template selection and progress tracking

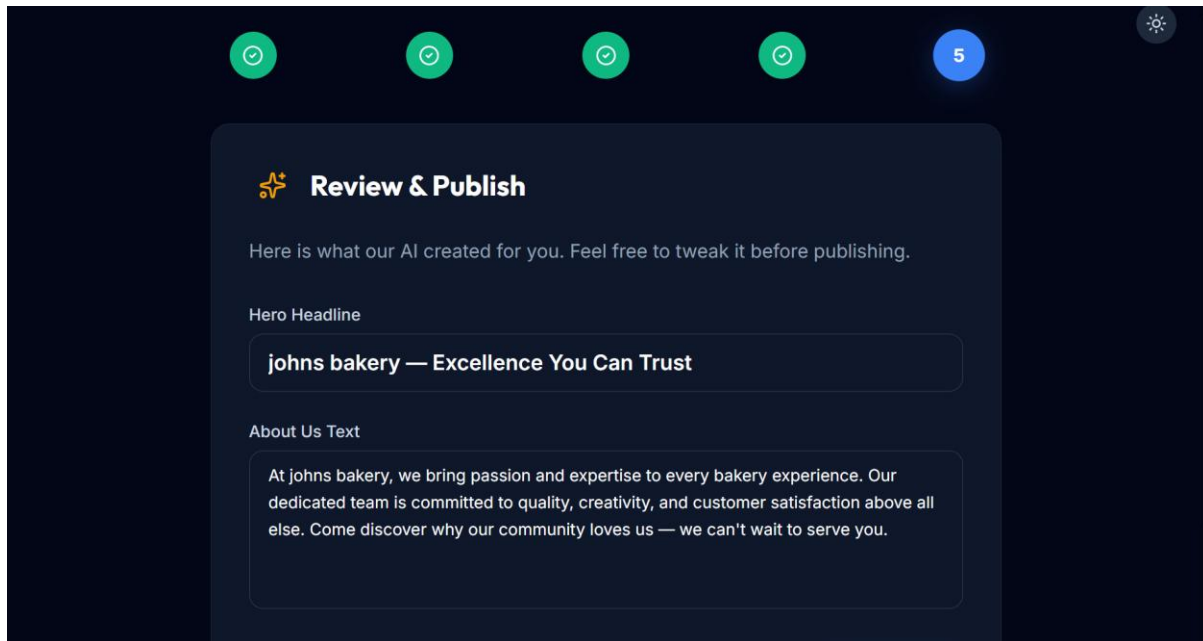
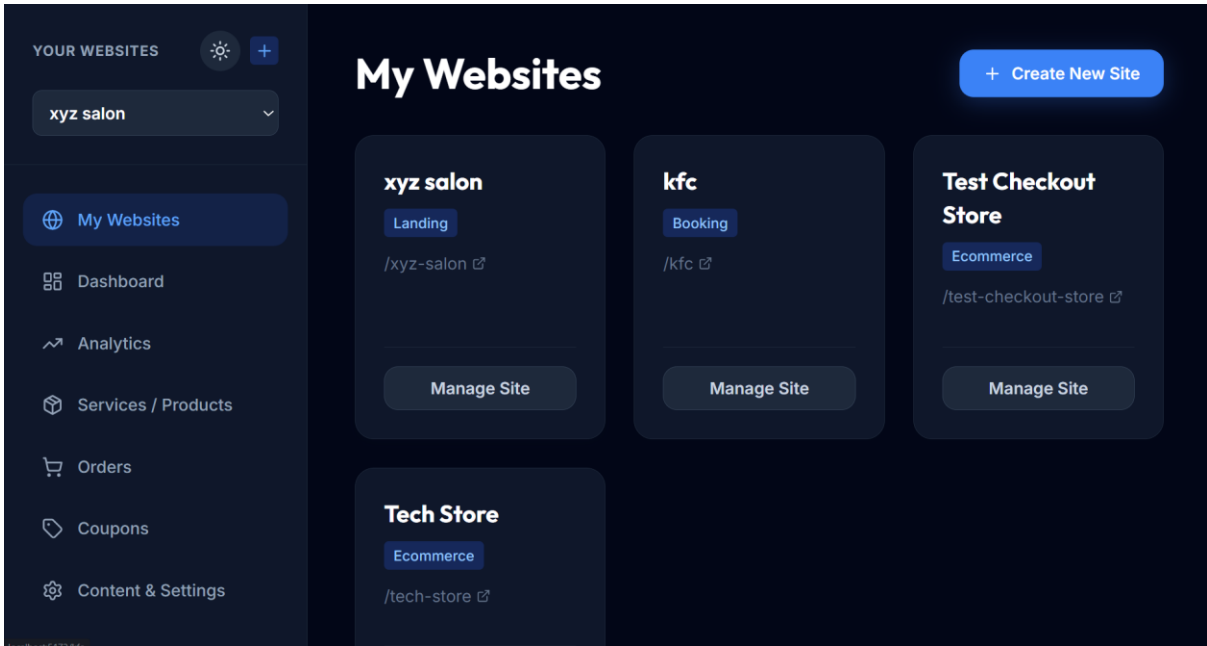


Figure 7.4 - Onboarding Wizard Step 5 displaying AI-generated marketing content ready for review

7.1.4 Admin Dashboard — Overview and Analytics

The Admin Dashboard (AdminDashboard.jsx) is the central control panel for authenticated tenants. The implementation utilizes a classic sidebar-and-main-content layout, which collapses into a hamburger menu or stacked layout on mobile devices.

The "Dashboard" and "Analytics" tabs provide a high-level overview of the business's performance. The UI employs "flat card" designs (subtle borders, no heavy drop shadows) to display key metrics such as Total Page Views, Total Orders, and Total Revenue. These values are dynamically calculated by reducing the arrays of orders and querying the pageViews counter from the tenant's MongoDB document. The dashboard also includes a prominent banner at the top displaying the tenant's live public URL with a direct link to visit the site.



Suggested Caption: Figure 7.5 - Admin Dashboard Overview displaying key business metrics and live URL

7.1.5 Admin Dashboard — Catalogue and Order Management

The operational management interfaces within the dashboard allow tenants to perform CRUD operations on their data.

- **Items Tab:** Displays a list of products or services. The implementation uses a modal window for creating and editing items, featuring form inputs for title, description, price, and stock count. Items with low or zero stock visually display warning badges (orange for low stock, red for out of stock).
- **Orders Tab:** Implements a responsive data table displaying all incoming customer orders. The table includes visual status badges (emerald for Pending, green for Delivered, red for Cancelled) and inline action buttons allowing the admin to update the order status with a single click. The "Export CSV" functionality is implemented via a dynamic Blob creation process that triggers a browser download.

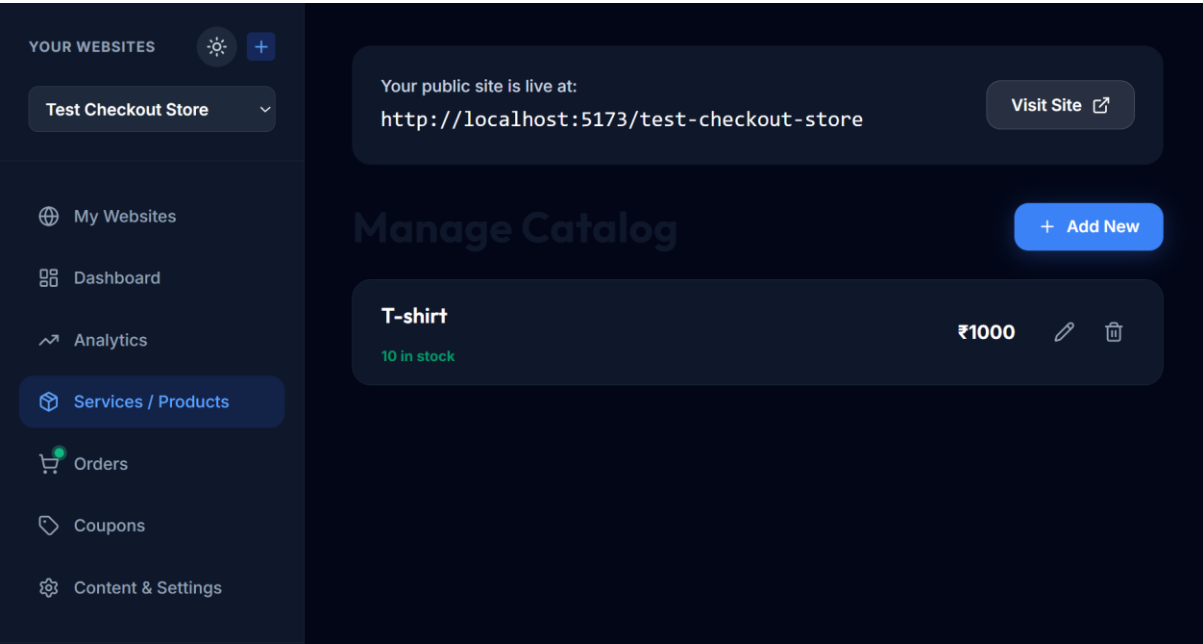


Figure 7.6 - Catalogue Management interface for adding and editing products/services

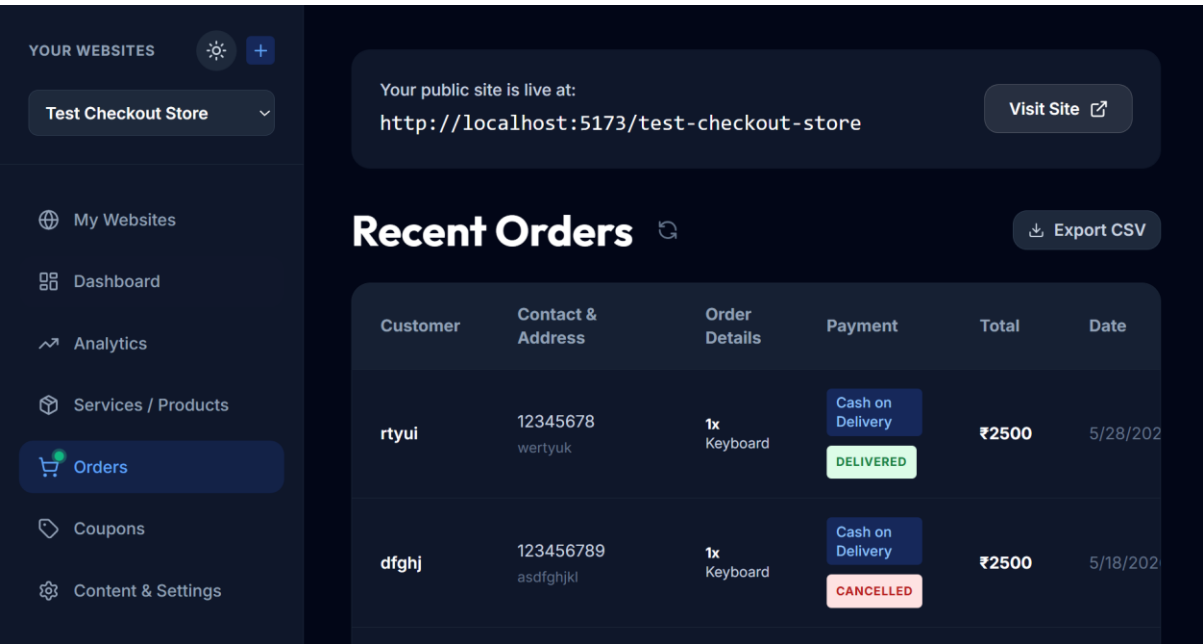


Figure 7.7 - Order Management interface displaying customer orders in a tabular format)

7.1.6 Dynamic Storefront — E-Commerce Template

The `TemplateA_Ecommerce.jsx` component implements the digital storefront for retail businesses. The interface is highly dynamic, relying heavily on React props passed down from the `SiteRenderer`. The tenant's chosen `primaryColor` is applied programmatically via inline styles to all CTA buttons, cart badges, and price highlights.

The layout features a hero section displaying the background image, AI-generated headline, and about text. Below the hero, the product catalogue is rendered using CSS Grid, creating a responsive layout that adapts from 1 column on mobile to 3 columns on desktop. Each product card includes an "Add to Cart" button that dynamically transforms into a quantity selector (- 1 +) when the item is added to the cart, automatically respecting the database-defined `stockCount` limits.

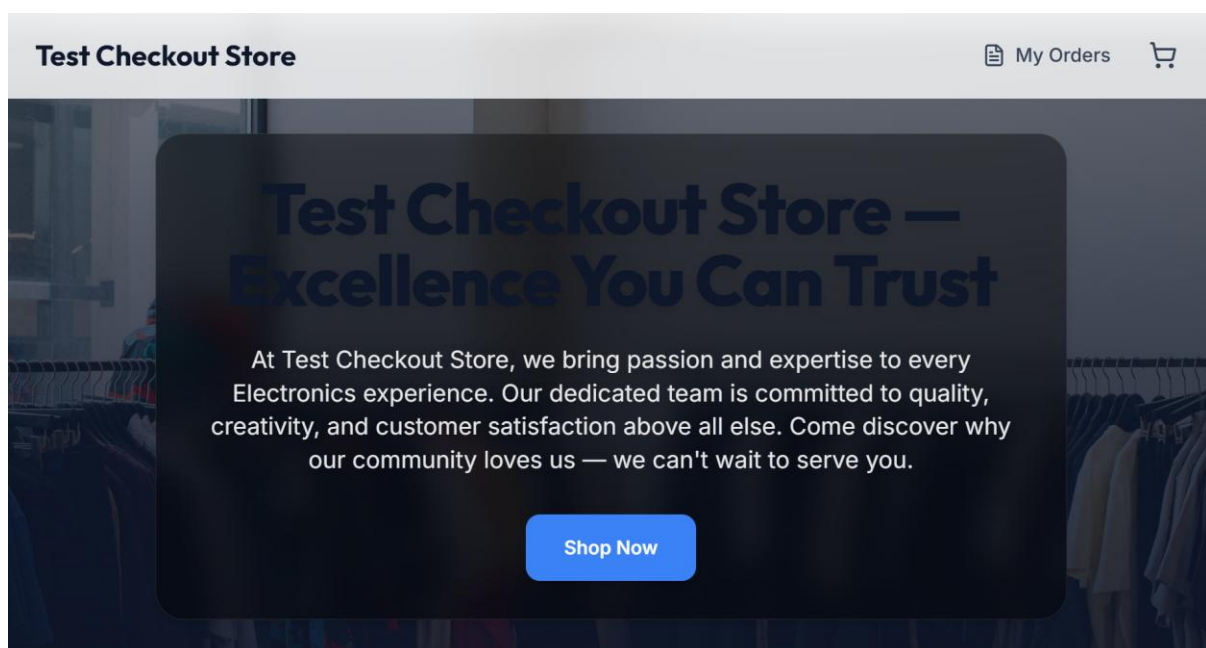
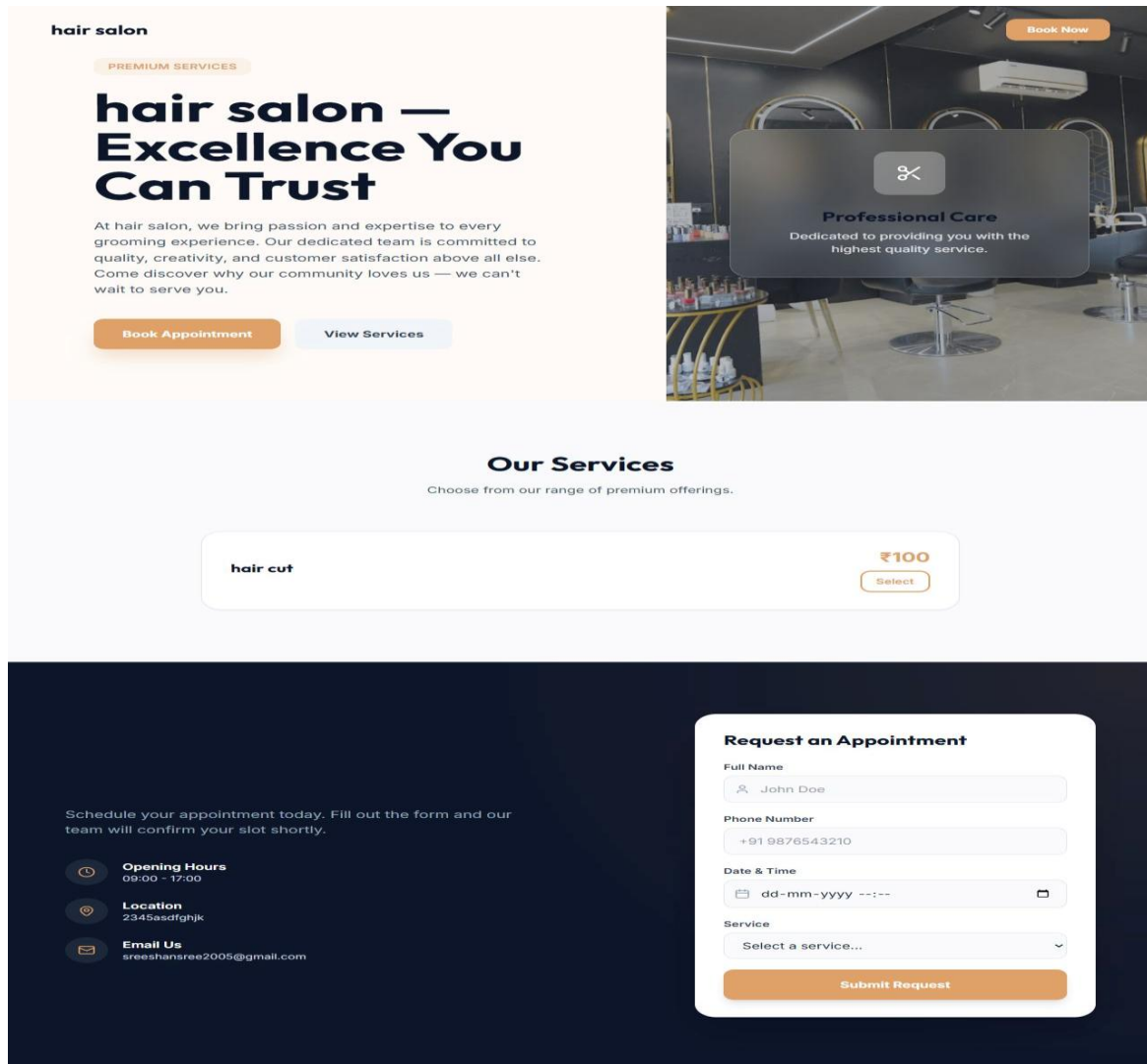


Figure 7.8 - Rendered E-Commerce Storefront customized with tenant branding and AI content

7.1.7 Dynamic Storefront — Booking and Service Template

The `TemplateB_Booking.jsx` component caters to service-oriented businesses. The visual implementation utilizes a split-screen design on desktop, pairing the textual hero content with an abstract graphic or image on the right.

Instead of a traditional shopping cart, this template lists services in a streamlined menu format. The primary call-to-action directs users to a comprehensive booking form located at the bottom of the page. This form incorporates native HTML5 datetime-local inputs and dropdown selects for choosing the specific service. The design emphasizes trust and clarity, prominently displaying the business's operating hours and physical location in an adjacent information panel.



© 2026 hair salon. Powered by BizWiz.

Figure 5.9 - Rendered Service & Booking Storefront featuring a split-screen hero design)

7.1.8 Customer Checkout and Cart Interface

The shopping cart and checkout flow (integrated into the E-Commerce template) are implemented as sliding sidebars and modal overlays to maintain context without navigating the user away from the storefront.

When the cart icon is clicked, a sidebar slides in from the right (animate-slide-left), displaying the selected items, calculated subtotal, and an input field for applying promotional coupons. The coupon application triggers an asynchronous validation request and instantly recalculates the total price, applying green visual feedback upon success.

The final checkout modal requires the customer to input their delivery details and select a payment method. The payment selection is implemented using large, clickable card elements rather than standard radio buttons, improving usability on touch devices. Upon submission, the UI transitions to a success screen featuring a green checkmark animation and displaying the newly generated Order ID.

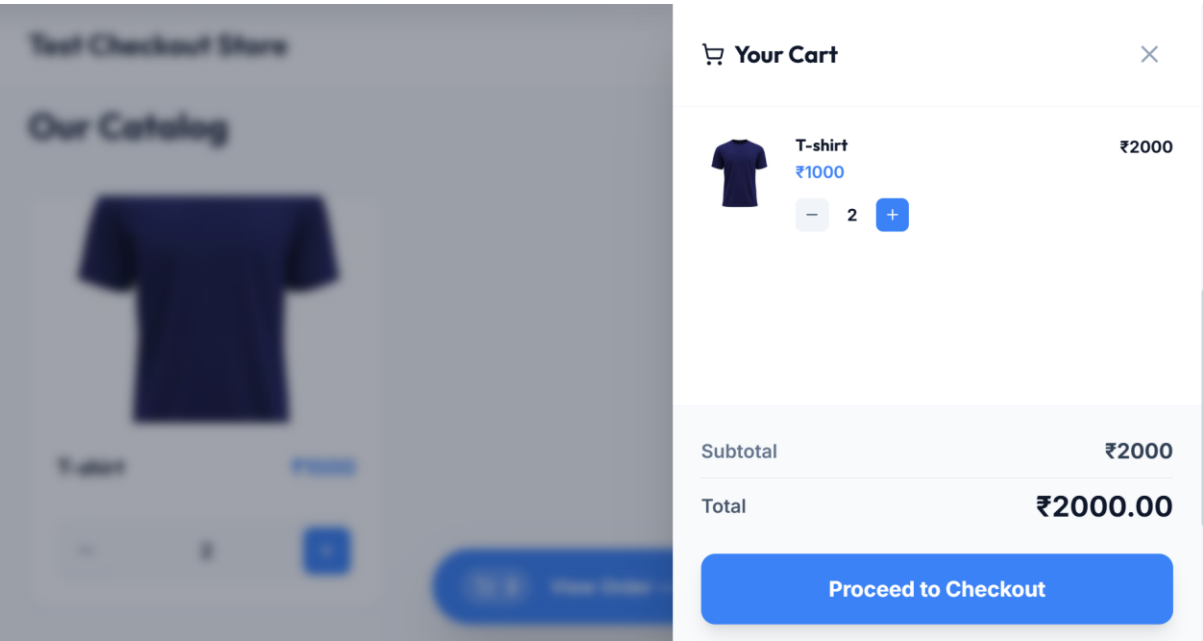
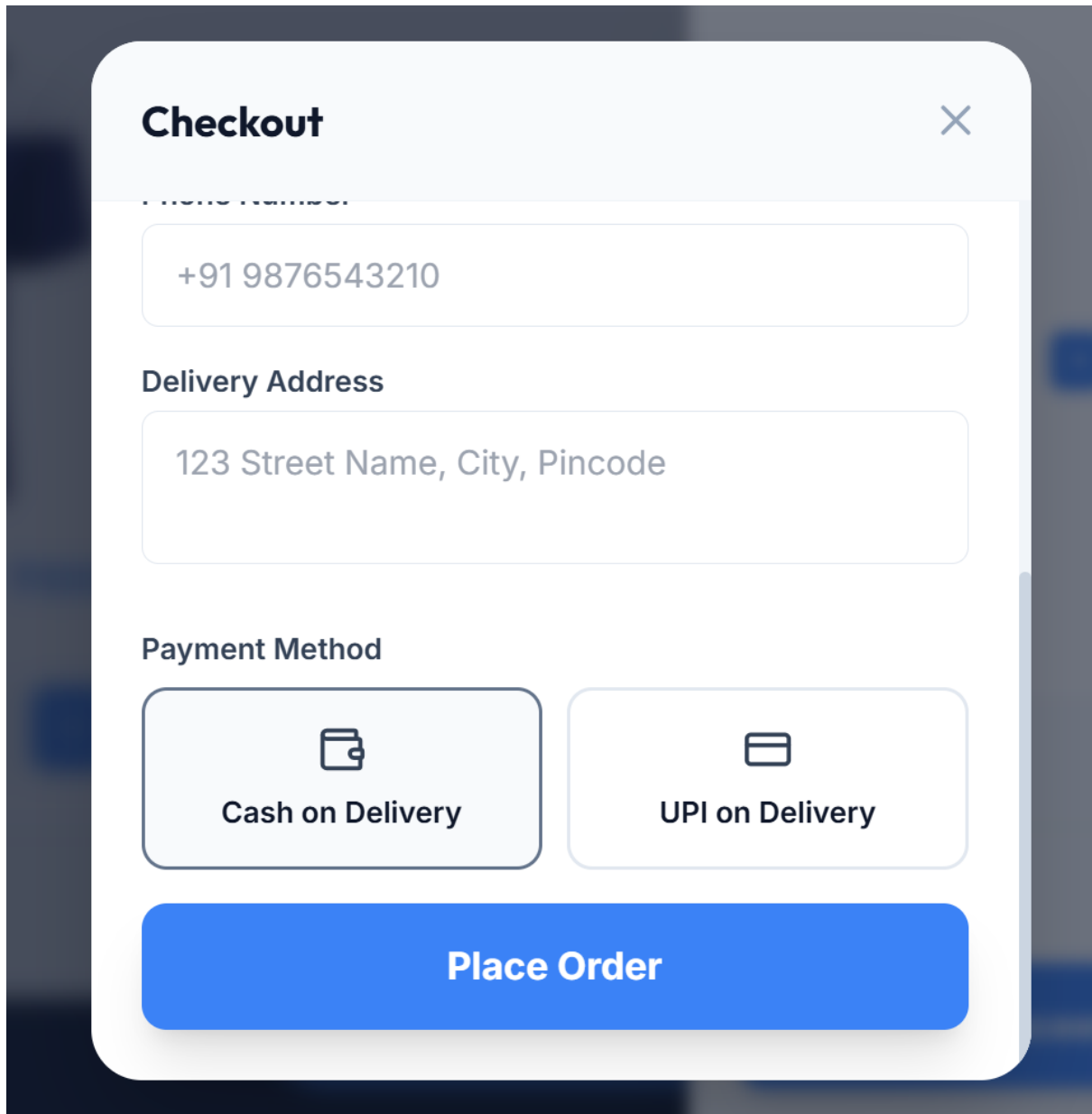


Figure 7.10 - Shopping Cart sidebar showing order summary and coupon application

A checkout modal window with a light gray background and rounded corners. At the top left is the title "Checkout" in bold black font, and at the top right is a close button represented by a gray "X" icon. Below the title is a section for "Phone Number" with a text input field containing "+91 9876543210". This is followed by a "Delivery Address" section with a text input field containing "123 Street Name, City, Pincode". Below the address is a "Payment Method" section containing two buttons: "Cash on Delivery" with a wallet icon and "UPI on Delivery" with a UPI icon. At the bottom of the modal is a large blue button with the text "Place Order" in white.

Checkout ×

Phone Number

+91 9876543210

Delivery Address

123 Street Name, City, Pincode

Payment Method

 **Cash on Delivery**

 **UPI on Delivery**

Place Order

Figure 7.11 - Checkout Modal displaying payment options and order confirmation

CHAPTER 8

FUTURE ENHANCEMENT

8.1 FUTURE ENHANCEMENTS

While the current implementation of the BizWiz multi-tenant SaaS platform successfully achieves its primary objectives of providing an AI-powered, low-barrier entry point for small businesses to establish an online presence, software development is an iterative process. The architecture has been deliberately designed to be extensible, allowing for the seamless integration of new features as the user base grows and business requirements evolve. The following enhancements are planned for future iterations of the platform.

8.1.1 Advanced AI Integrations

The current integration with Google Gemini 1.5 Flash provides automated hero headlines and "About Us" text during the onboarding process. Future updates will expand this AI capability significantly:

1. **AI-Powered Product Descriptions:** When a tenant admin adds a new product or service, they will only need to provide a basic title (e.g., "Vanilla Cake"). A new "Generate Description" button will invoke the AI to create a compelling, SEO-optimized product description, saving the business owner significant time and improving their storefront's search engine ranking.
2. **Intelligent Image Generation:** Currently, tenants must provide their own image URLs. Future versions will integrate with image generation models (such as DALL-E 3 or Google Imagen) to automatically generate professional placeholder images for products and background hero banners based on the tenant's business niche.
3. **Chatbot Integration:** A tenant-specific, AI-powered customer support chatbot could be embedded into the storefront templates. By feeding the tenant's product catalog and business hours into the AI prompt context, the chatbot could answer basic customer inquiries and assist with the booking process autonomously.

8.1.2 Custom Domain Mapping

Currently, all tenant storefronts are hosted on the BizWiz subdomain structure (e.g., bizwiz.com/johns-bakery). While this is sufficient for getting started, established businesses often prefer to use their own custom domains (e.g., www.johnsbakery.in) for brand credibility.

The planned implementation for custom domain mapping involves:

1. Updating the Tenant MongoDB schema to include a customDomain field.
2. Implementing a reverse proxy (using Nginx or a cloud edge service like Cloudflare Workers) that intercepts incoming requests, checks the host header against the database, and routes the request to the appropriate tenant's React application.
3. Automating SSL certificate provisioning for custom domains using Let's Encrypt API integrations.

8.1.3 Integrated Payment Gateways

The current order processing system supports "Cash on Delivery," "UPI on Delivery," and "Pay at Venue" options. While these are popular in the Indian market, integrating digital payment gateways is essential for scaling e-commerce operations.

Future development will integrate a payment aggregator (such as Razorpay or Stripe) directly into the checkout flow. Due to the multi-tenant nature of the platform, this will require a split-payment architecture (e.g., Razorpay Route or Stripe Connect) where the customer's payment is processed by BizWiz, a small platform transaction fee is optionally deducted, and the remaining funds are automatically routed to the individual tenant's connected bank account.

8.1.4 Advanced Analytics and Marketing Tools

The current dashboard tracks basic metrics such as page views, order volume, and revenue. Future iterations will introduce a more comprehensive analytics suite:

1. **Conversion Tracking:** Tracking the funnel from page view, to adding an item to the cart, to completing the checkout. This will help tenants identify where they are losing potential customers.

2. **Customer CRM:** A dedicated Customer Relationship Management tab in the dashboard that automatically builds profiles of repeat customers based on their order history.
3. **Automated Email Marketing:** Integration with services like SendGrid or AWS SES to allow tenants to send automated order confirmation emails, abandoned cart reminders, and promotional newsletters using their active coupon codes.

8.2 SCALABILITY AND DEPLOYMENT PLAN

As the platform acquires more tenants, the underlying infrastructure must scale to maintain performance and reliability. The current MERN stack architecture is inherently scalable, but the deployment strategy will evolve through the following phases.

8.2.1 Containerization and Orchestration

The application will be migrated from standard virtual machine or PaaS hosting to a containerized environment using Docker. Both the React frontend (served via Nginx) and the Node.js backend will be packaged into lightweight Docker images.

For orchestration, Kubernetes (K8s) will be implemented to manage these containers. Kubernetes will allow the platform to automatically scale the number of Node.js backend pods up or down based on real-time CPU utilization and incoming HTTP traffic, ensuring that sudden spikes in storefront visitors do not degrade the experience for other tenants.

8.2.2 Database Sharding and Caching

While MongoDB Atlas currently handles the database load, a massive influx of tenants will require a robust data strategy:

1. **Redis Implementation:** A Redis in-memory data store will be introduced to cache frequently accessed, rarely changing data. For example, a tenant's configuration (theme type, colors, business name) will be cached in Redis. When an end-user visits the storefront, the SiteRenderer will fetch data from the ultra-fast Redis cache rather than querying MongoDB, drastically reducing database load and improving page load speeds.

2. **Database Sharding:** If the tenants collection grows beyond the capacity of a single replica set, MongoDB sharding will be implemented. The database could be sharded based on geographic region or a hashed tenantId, distributing the storage and read/write load across multiple physical servers.

8.2.3 Content Delivery Network (CDN)

To ensure fast storefront loading times for customers regardless of their geographic location, all static assets (React JavaScript bundles, CSS files, and platform images) will be served through a global Content Delivery Network (CDN) such as Cloudflare or AWS CloudFront. Furthermore, the CDN will provide DDoS protection and Web Application Firewall (WAF) capabilities, shielding the backend API from malicious traffic.

CHAPTER 9

CONCLUSION

The development and deployment of the BizWiz multi-tenant SaaS platform represent a significant step forward in democratizing digital commerce for small and medium-sized enterprises (SMEs). Throughout this project, the primary objective was to eliminate the technical and financial barriers that traditionally prevent local businesses, freelancers, and independent service providers from establishing a professional, fully functional online presence. By leveraging modern web technologies and generative artificial intelligence, BizWiz has successfully transformed a complex, multi-week development process into a seamless, five-minute onboarding experience.

From a technical perspective, the project successfully demonstrated the viability and efficiency of a true multi-tenant architecture. By utilizing a single shared application instance (Node.js/Express) and a single shared database (MongoDB Atlas) with strict logical data isolation, the platform achieved high scalability and resource efficiency. This architectural decision ensures that the cost of serving each additional tenant approaches zero, making a free or low-cost SaaS business model economically sustainable. The implementation of the embedded document pattern in MongoDB proved highly effective, allowing complex tenant storefronts—complete with branding configurations, product catalogs, and active coupon codes—to be fetched and rendered in a single, performant database query.

The frontend implementation, utilizing React.js, Vite, and Tailwind CSS, delivered a fast, responsive, and intuitive user experience. The dynamic rendering engine (`SiteRenderer`) successfully bridged the gap between a centralized backend and decentralized, tenant-specific storefronts. By dynamically injecting database-driven configurations into pre-built React components, the system proved that high levels of customization (colors, fonts, layouts, and SEO metadata) can be achieved without requiring individual codebases for each business. Furthermore, the integration of Google OAuth 2.0 ensured a secure, passwordless

authentication flow that protected both tenant administrators and the platform's overall integrity.

The most innovative aspect of the BizWiz platform proved to be its integration with the Google Gemini 1.5 Flash AI model. By automating the generation of marketing copy (hero headlines and business descriptions), the platform solved one of the most persistent challenges faced by non-technical business owners: content creation. The AI integration was implemented seamlessly, with robust fallback mechanisms ensuring that network timeouts or API failures would never block a user from completing their setup.

In conclusion, the BizWiz project successfully met all of its functional and non-functional requirements. It provides a comprehensive, end-to-end solution that empowers business owners to take control of their digital brand identity without relying on expensive developers or rigid third-party marketplaces. The platform stands as a robust foundation that is ready for real-world usage and is architecturally prepared for future scaling, custom domain integrations, and advanced e-commerce features.

CHAPTER 10

REFERENCES

The development of the BizWiz multi-tenant SaaS platform relied on comprehensive documentation, academic literature, and official technical resources for the MERN stack, generative AI, and cloud architecture. The following references were instrumental in the system's design, implementation, and testing phases.

1. **React.js Official Documentation.** Meta Platforms, Inc. (2024). *React: The library for web and native user interfaces*. Retrieved from <https://react.dev/> (Utilized for understanding component lifecycle, React Hooks (*useState*, *useEffect*), Context API for state management, and modern functional component architecture.)
2. **Node.js API Reference.** OpenJS Foundation. (2024). *Node.js v16.x Documentation*. Retrieved from <https://nodejs.org/docs/latest-v16.x/api/> (Referenced for asynchronous programming models, event loop architecture, and server-side JavaScript execution.)
3. **Express.js Framework Guide.** OpenJS Foundation. (2024). *Express - Node.js web application framework*. Retrieved from <https://expressjs.com/> (Consulted for REST API design, middleware implementation, CORS configuration, and route handling logic.)
4. **MongoDB and Mongoose Documentation.** MongoDB, Inc. (2024). *MongoDB Atlas and Mongoose ODM v8.x*. Retrieved from <https://mongoosejs.com/docs/guide.html> (Used extensively for designing the embedded document schemas, understanding NoSQL query performance, and implementing unique indexing strategies for tenant isolation.)
5. **Google Generative AI Documentation.** Google LLC. (2024). *Gemini API and Google AI Studio*. Retrieved from <https://ai.google.dev/docs> (Essential for implementing the @google/generative-ai SDK, understanding prompt engineering best practices for the Gemini 1.5 Flash model, and parsing JSON responses.)
6. **Google Identity Services.** Google LLC. (2024). *OAuth 2.0 for Web Server Applications*. Retrieved

from <https://developers.google.com/identity/protocols/oauth2> (Referenced for implementing the secure, passwordless authentication flow and verifying credential tokens on the backend using google-auth-library.)

7. **Tailwind CSS Documentation.** Tailwind Labs Inc. (2024). *Tailwind CSS - Rapidly build modern websites without ever leaving your HTML.* Retrieved from <https://tailwindcss.com/docs> (Consulted for utility-first responsive design principles, configuring dynamic themes, and implementing CSS grid/flexbox layouts across varying viewport sizes.)
8. **Vite Build Tool.** Evan You. (2024). *Vite - Next Generation Frontend Tooling.* Retrieved from <https://vitejs.dev/guide/> (Used to optimize frontend asset bundling, implement Hot Module Replacement (HMR), and configure environment variables securely.)
9. **JSON Web Token (JWT) Standard.** Internet Engineering Task Force (IETF). (2015). *RFC 7519: JSON Web Token (JWT).* Retrieved from <https://datatracker.ietf.org/doc/html/rfc7519> (Studied for understanding stateless session management, secure payload encoding, and HMAC SHA-256 signature verification.)
10. **Software Engineering: A Practitioner's Approach (8th Edition).** Pressman, R. S., & Maxim, B. R. (2014). McGraw-Hill Education. (Consulted for structural guidelines on writing Software Requirement Specifications (SRS), creating Data Flow Diagrams (DFD), and establishing comprehensive software testing methodologies.)